

Real-Time Model Monitoring & Alerting Stack

The Guidebook — Prometheus · Grafana · Alertmanager · Drift Exporter · MLflow · Docker Compose

Team 4 · MLOps · wrapped around the Telco Customer-Churn model

Concept → Why → Tools → Alternatives → Trade-offs → Why chosen → Implementation → PromQL/code → Demo → Troubleshooting → Q&A

Built 2026-08-18 · reproducible with `make up` & `make train`

Real-Time Model Monitoring & Alerting Stack — The Guidebook

Team 4 · MLOps · Telco Customer Churn model (from Team "customer-churn-mlops")

Why this book exists

Anyone can copy a `docker-compose.yml` from the internet and get Grafana to show a line. The point of this project — and of this book — is to be able to answer "**why?**" for every box in the architecture:

- why Prometheus and not Datadog / InfluxDB / ELK / OpenTelemetry?
- why a *histogram* and not a *summary*? why *p95* and not the *average*?
- why *pull* instead of *push*?
- why **PSI** and **KS**, and why on a *rolling window*?
- why Alertmanager and not Grafana's own alerting?
- why Docker Compose and not Kubernetes?
- what is the difference between *monitoring* and *observability*, *data drift* and *concept drift*, an *operational* alert and an *ML-quality* alert?

Every chapter follows the same pattern:

Concept → Why it matters → Available tools → Alternatives & trade-offs → Why we chose ours → How we implemented it → PromQL / code → How to demo it → Troubleshooting → Questions you may be asked

What we built (one paragraph)

A **model-agnostic observability stack**: any HTTP prediction service can be dropped behind a small *monitoring gateway* that exports the four operational signals (request rate, latency p50/p95/p99, error rate, prediction distribution) to **Prometheus**; a custom **drift exporter** compares a rolling window of live features/predictions against a training-time *reference profile* using **PSI** and the **KS-test**; **Grafana** shows four provisioned dashboards; **Prometheus alert rules** (operational + ML-quality) are routed by **Alertmanager** to Slack and e-mail; **MLflow** tracks experiments and holds the model registry; a **load generator** with switchable *scenarios* (drift, latency, errors, burst) makes every alert demonstrable live. Everything runs with one command: `make up`.

The sample model is the other team's *Customer Churn* GradientBoosting model (MLflow/skops artifact), served by their FastAPI code — with one bug fixed (see §4.7) and Prometheus metrics added.

How this maps to the assignment and to the whiteboard

Requirement (assignment)	Where
Prometheus exporters: request rate, latency p50/p95/p99, error rate, prediction distribution	<code>services/gateway/gateway.py</code> (model-agnostic) and <code>services/model_api/api.py</code> (in-process) — Ch. 4
Custom exporter for data drift (PSI or KS) on a rolling window	<code>services/drift_exporter/drift_exporter.py</code> — Ch. 5
Grafana dashboards + alert rules routed to Slack/e-mail via Alertmanager	<code>monitoring/grafana/</code> , <code>monitoring/prometheus/rules/alert_rules.yml</code> , <code>monitoring/alertmanager/</code> — Ch. 6, 7
Docker Compose stack (Prometheus + Grafana + Alertmanager + sample model API)	<code>docker-compose.yml</code> — Ch. 9
Documentation on wiring the stack to any new model	<code>docs/wiring-guide.md</code> — Ch. 10

Whiteboard item (instructor)	Where in this book
Prometheus: check target health · rate of change · average over time · time ranges 5m/1h/1d/1mo · graph vs table	§3.6 PromQL cookbook
Grafana: edit a panel · create a panel · set alert thresholds · change time range · change refresh rate	§6.4 hands-on
MLflow: compare runs · filter by params/metrics · visualise hyper-parameter effects · inspect best model · register a model · export CSV	§8.4 hands-on

Table of contents

1. Monitoring & Observability for ML systems

2. Architecture of the stack
 3. Prometheus & PromQL
 4. Exporters — the four operational signals
 5. Drift — PSI, KS and the rolling window
 6. Grafana — dashboards
 7. Alerting — Prometheus rules + Alertmanager
 8. MLflow — experiment tracking & model registry
 9. Docker Compose — orchestration
 10. Wiring the stack to a new model
 11. Demo script
 12. Troubleshooting
 13. Questions the examiner may ask (with answers)
 14. Glossary & references
-

1 · Monitoring & Observability for ML systems

1.1 Concept

Monitoring is watching *known* signals against *known* thresholds ("is p95 latency under 500 ms?", "is the service up?"). You decide in advance what to look at.

Observability is the property of a system that lets you answer *questions you did not anticipate* from the data it emits — logs, metrics and traces (the "three pillars"). Monitoring is something you *do*; observability is something a system *has*. A well-instrumented system with rich labels (model, version, feature, status) lets you slice a problem in ways nobody planned for.

Our stack is a **metrics-first observability stack** with monitoring (dashboards + alerts) built on top.

1.2 Why an ML system needs *more* than ordinary monitoring

A normal web service fails loudly: 500s, timeouts, crashes. An ML service can be **perfectly healthy operationally and completely wrong**: 200 OK, 20 ms latency, and predictions that are garbage because the world changed (or a preprocessing bug scrambled the features — exactly what happened in the model we wrapped, §4.7). So we monitor two families of signals:

Family	Question	Signals	Alerts
Operational ("golden signals")	Is the service healthy?	request rate, latency, error rate, saturation, up/down	ModelServiceDown, HighLatencyP95, HighErrorRate, TrafficSpike, NoTraffic
ML quality	Is the <i>model</i> still valid?	prediction distribution, positive-rate, feature drift (PSI/KS), prediction drift, (later: accuracy when labels arrive)	FeatureDriftHigh, MultipleFeaturesDrifted, PredictionDrift, PositiveRateAnomaly

The four **golden signals** (Google SRE book): *latency, traffic, errors, saturation*. Our "prediction distribution" is the ML-specific fifth signal the assignment asks for.

1.3 SLI · SLO · SLA — the vocabulary behind thresholds

- **SLI** (indicator): a measured quantity — "p95 latency over 5 min", "share of non-5xx responses".
- **SLO** (objective): the target — "p95 < 500 ms", "99.5 % availability over 30 days".
- **SLA** (agreement): the contract with consequences if the SLO is missed.

Our recording rules (`monitoring/prometheus/rules/recording_rules.yml`) compute the SLIs (`ml:latency_p95:5m`, `ml:error_ratio:5m`, `ml:availability:30m`); the alert rules encode the SLOs.

1.4 Metrics vs logs vs traces — why metrics first

Pillar	Good at	Bad at	Cost
Metrics (numbers over time)	trends, alerting, dashboards, cheap long retention	the <i>why</i> of a single request	very low
Logs (events)	debugging one request, audit	aggregation at scale, cost	high
Traces (spans across services)	latency breakdown across micro-services	not for alerting	medium

For a monitoring & alerting stack, metrics are the natural backbone: cheap, aggregatable, and what alerting engines evaluate. Logs/traces (Loki, Tempo, Jaeger, OpenTelemetry) are the natural *next* addition — see §14.

1.5 Push vs pull (preview)

Prometheus **pulls** ("scrapes") `/metrics` from every service on a schedule. Alternatives such as StatsD/Graphite, InfluxDB-Telegraf, Datadog agents *push*. Why pull won for us is in §3.3.

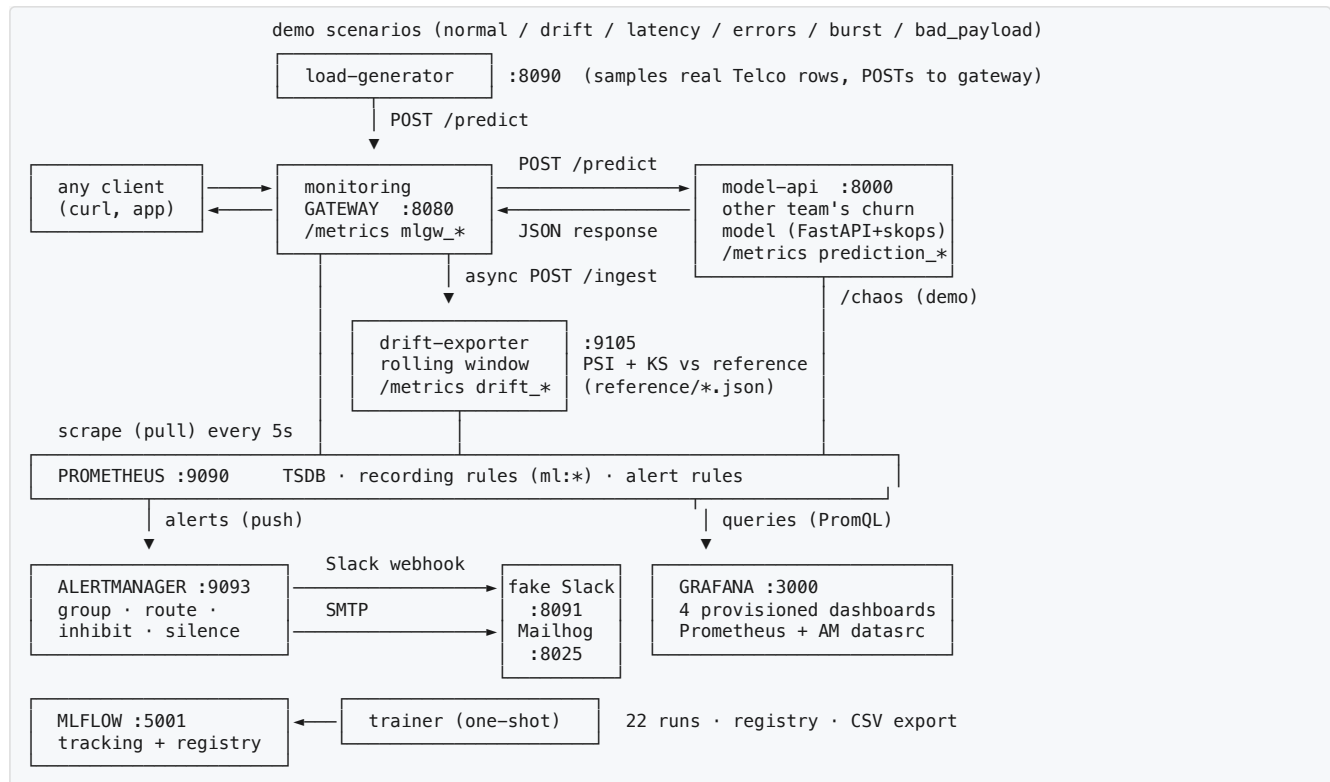
1.6 Questions you may be asked

- *"Monitoring vs observability?"* — monitoring = known unknowns with thresholds; observability = being able to ask new questions from emitted telemetry. Labels + PromQL give us the latter.
- *"Why is a model harder to monitor than a web service?"* — silent failure: healthy HTTP, wrong answers. Hence prediction distribution + drift metrics.
- *"What is a golden signal?"* — latency, traffic, errors, saturation (Google SRE).

- "*Where does accuracy monitoring fit?*" — needs ground-truth labels, which arrive late (churn is known months later). Drift is the *leading* indicator; accuracy is the *lagging* one. Our stack monitors the leading indicators in real time and is designed so an "accuracy exporter" can be added the same way the drift exporter was (§10).
-

2 · Architecture of the stack

2.1 The picture



Every box is a container in `docker-compose.yml`; every arrow marked *scrape* is Prometheus pulling `/metrics` (Prometheus also scrapes itself, Alertmanager, Grafana and MLflow — "meta-monitoring").

2.2 Data flow for one prediction

1. Client → **gateway** `POST /predict` with the raw feature JSON.
2. Gateway forwards the body unchanged to `UPSTREAM_URL + PREDICT_PATH`, timing it.
3. Response comes back; gateway records `mlgw_requests_total{status}`, `mlgw_request_latency_seconds` (histogram), extracts the score (`PROB_FIELD`) and class (`LABEL_FIELD`) → `mlgw_prediction_score` (histogram) and `mlgw_predictions_total{predicted_class}`.
4. Gateway *asynchronously* posts `{features, prediction, score}` to the **drift exporter** `/ingest` (fire-and-forget: drift must never slow down predictions).
5. Drift exporter appends the row to a `deque(maxlen=WINDOW_SIZE)`; every `COMPUTE_INTERVAL` seconds it recomputes PSI/KS for every feature against the reference profile and updates gauges.
6. Prometheus scrapes gateway, model-api and drift-exporter every 5 s; evaluates recording rules (SLIs) and alert rules; firing alerts are pushed to Alertmanager.
7. Alertmanager groups/de-duplicates/inhibits and notifies Slack / e-mail.
8. Grafana queries Prometheus for the dashboards; the "Critical alerts" annotation overlays firing alerts on every graph.

2.3 Key design decisions (and the alternatives we rejected)

Decision	Chosen	Alternatives	Why
Where to measure	Sidecar/proxy gateway in front of the model + optional in-process library	(a) only in-process instrumentation; (b) service mesh (Istio/Linkerd)	(a) needs code changes in every model and every language; (b) is heavy and gives no ML semantics (score, class). The gateway is ~150 lines, language-agnostic, needs only env vars, and <i>also</i> observes the model from the outside (network + serialization included). We keep the in-process metrics in <code>model_api</code> to show the alternative and to compare the two (\$4.5).
Metrics backend	Prometheus	InfluxDB, Graphite, Datadog, CloudWatch, VictoriaMetrics, OpenTelemetry Collector + backend	\$3.2

Decision	Chosen	Alternatives	Why
Collection model	pull (scrape)	push (StatsD, Pushgateway, OTLP)	§3.3
Latency metric type	Histogram	Summary	§3.5 — histograms aggregate across instances and let <i>Prometheus</i> compute any quantile.
Drift statistic	PSI for every feature + KS for numeric features + PSI on scores	only KS, only PSI, Jensen–Shannon, Wasserstein, χ^2 , classifier-based drift	§5.3
Drift computation	separate exporter, rolling window in memory, reference JSON	inside the gateway; batch job (Evidently report every hour); external SaaS	keeps the gateway stateless; "custom exporter" is what the assignment asks; real-time.
Alerting engine	Prometheus rules → Alertmanager	Grafana-managed alerts; SaaS pagers	§7.2
Dashboards	Grafana, provisioned as code (generated JSON)	hand-clicked dashboards; Kibana; Datadog	§6.2
Orchestration	Docker Compose	Kubernetes (+ kube-prometheus-stack), bare processes	§9.2
Experiment tracking	MLflow	W&B, Neptune, Comet, DVC, SageMaker	§8.2

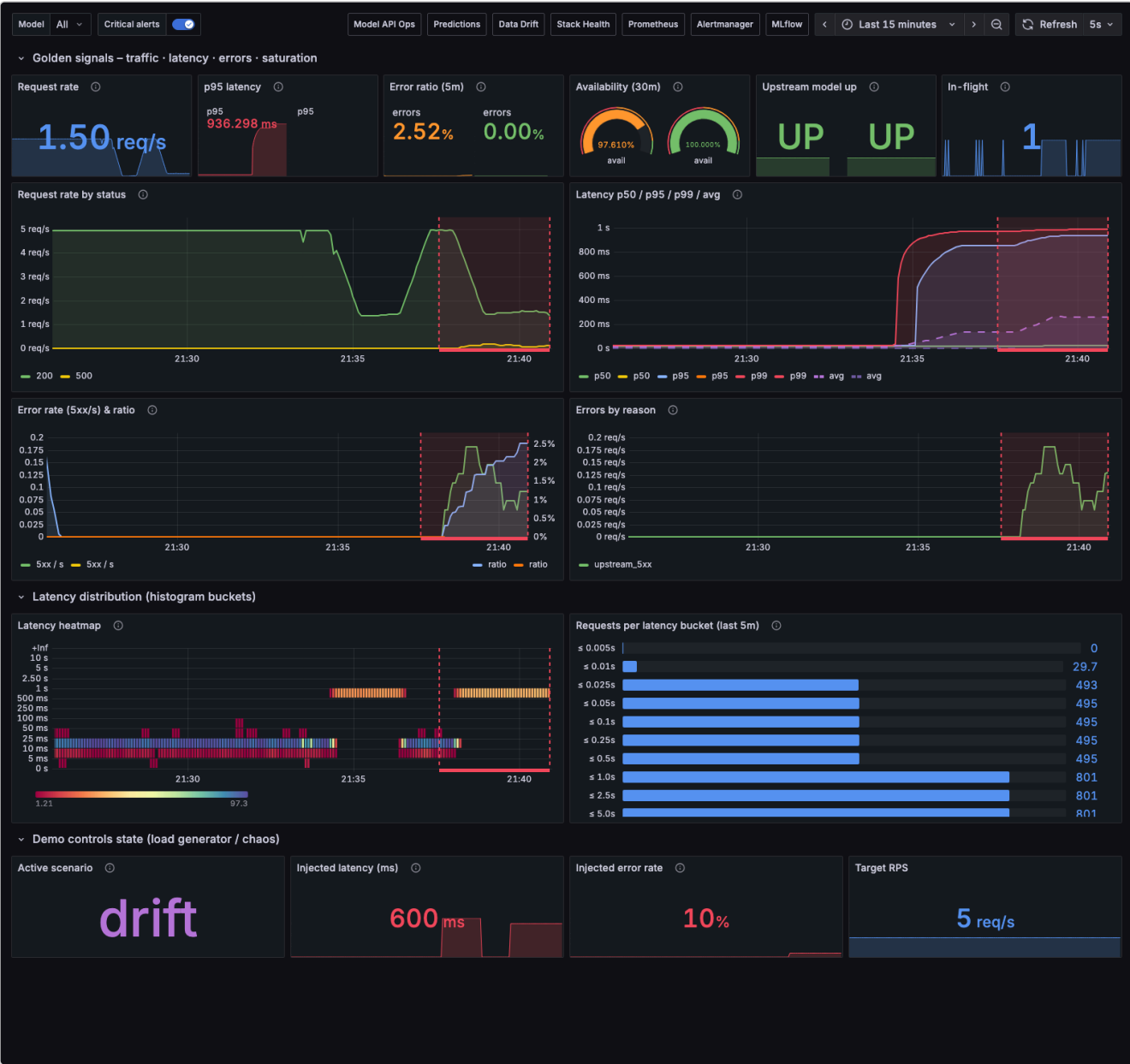
2.4 What makes it "reusable / model-agnostic"

- The gateway knows nothing about churn. Point `UPSTREAM_URL` at any model, name the JSON fields that hold score/class, done. `docker-compose.second-model.yml` proves it by monitoring the other team's *original, unmodified* image next to ours.
- Every metric carries a `model` label → one Prometheus, one Grafana, N models. Dashboards have a `$model` variable.
- The drift exporter takes any reference JSON (built with `build_reference.py` from any CSV).
- Alerts are written against the gateway's `mlgw_*` metrics and recording rules, so they apply to every model automatically.

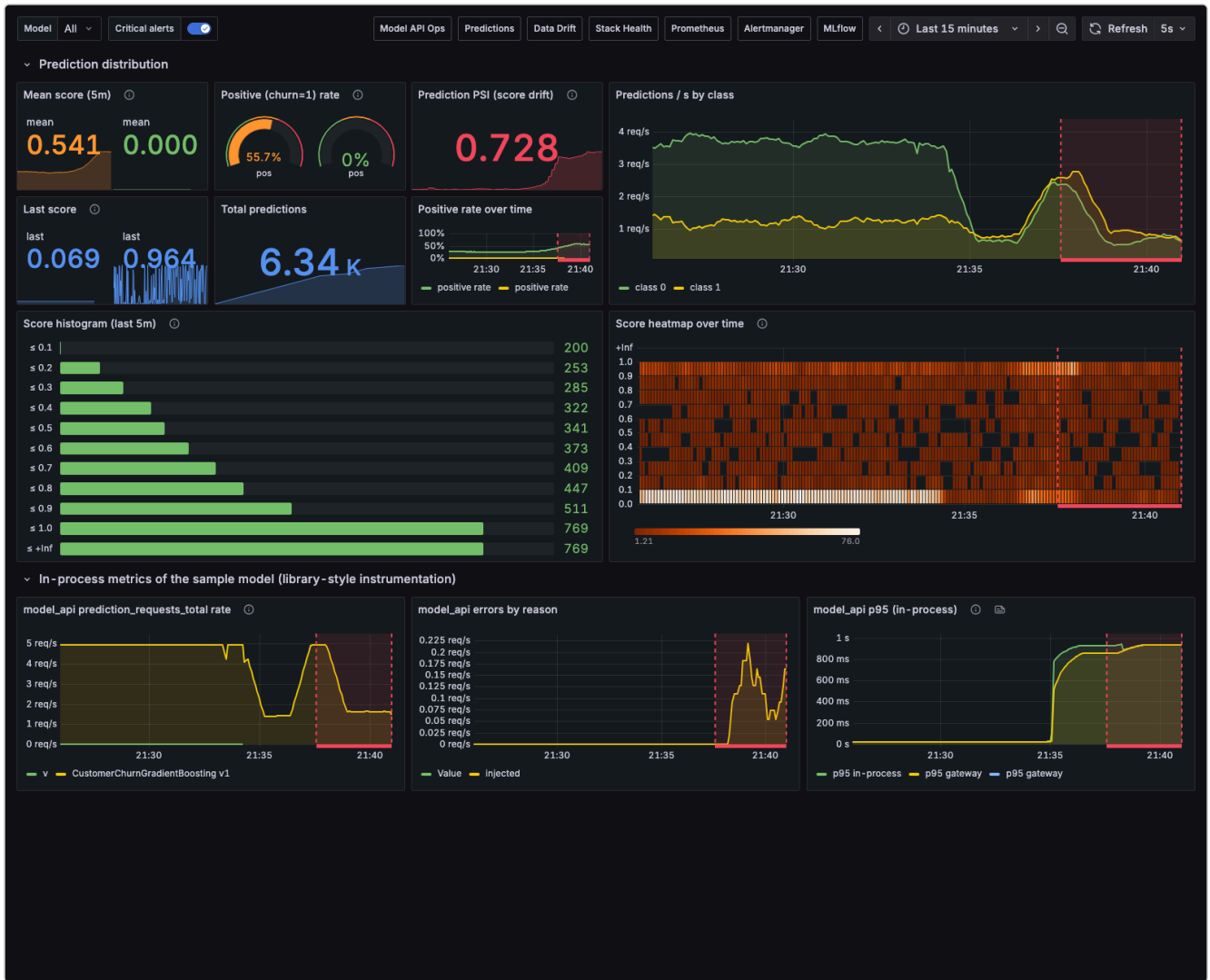
2.5 Ports cheat-sheet

Service	URL	Login
Grafana	<code>http://localhost:3000</code>	admin / admin
Prometheus	<code>http://localhost:9090</code>	—
Alertmanager	<code>http://localhost:9093</code>	—
MLflow	<code>http://localhost:5001</code> (5000 is used by macOS AirPlay)	—
Model API (docs)	<code>http://localhost:8000/docs</code>	—
Gateway	<code>http://localhost:8080/metrics</code>	—
Drift exporter	<code>http://localhost:9105/metrics</code>	—
Load generator	<code>http://localhost:8090/docs</code>	—
Mailhog (e-mail inbox)	<code>http://localhost:8025</code>	—
Fake Slack (webhook catcher)	<code>http://localhost:8091</code>	—

Screenshots (captured from the running stack)



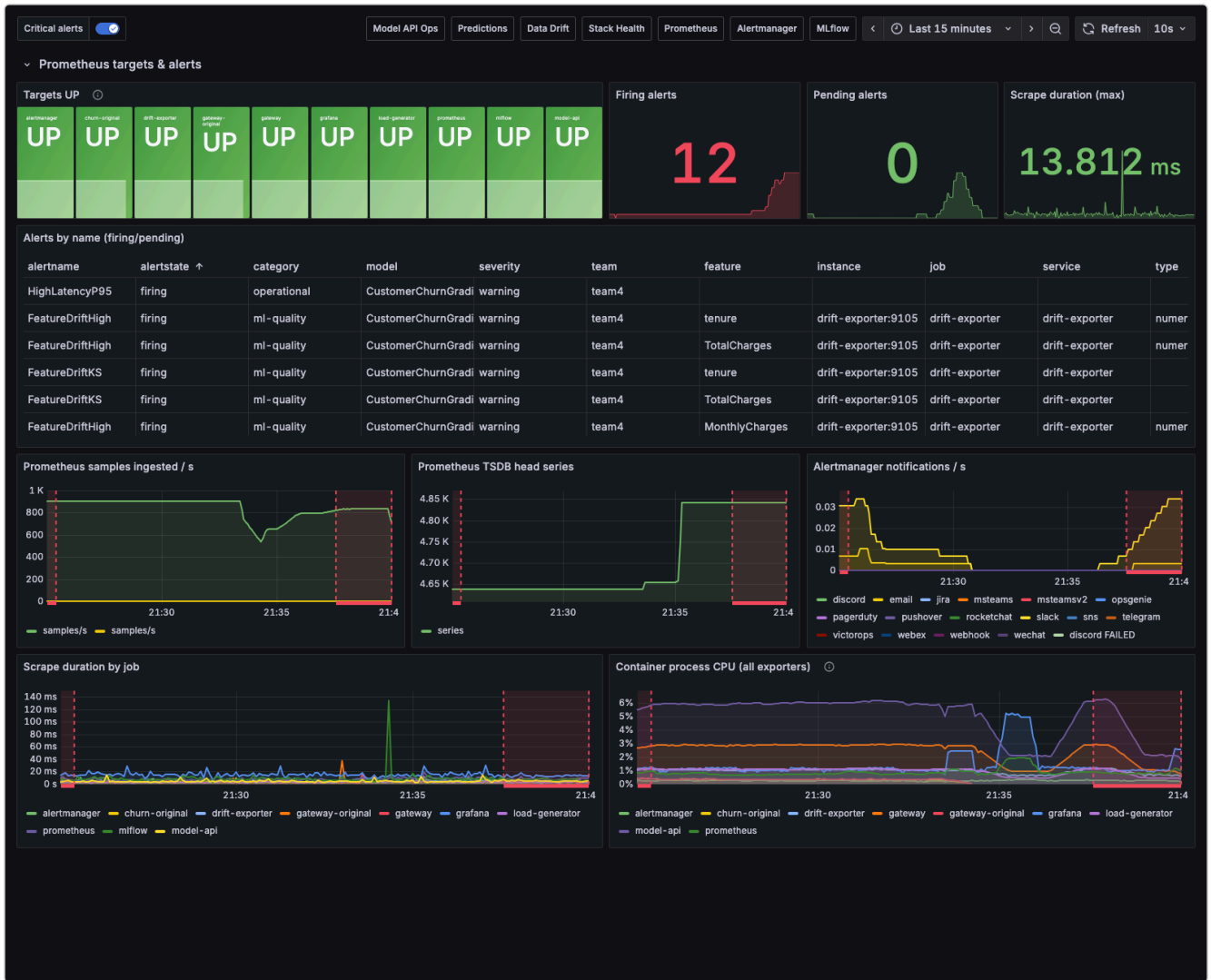
Dashboard 1 · Model API – Operational (golden signals) during the latency scenario



Dashboard 2 - Predictions – score distribution, positive rate, in-process vs gateway



Dashboard 3 - Data Drift - PSI/KS on the rolling window during the drift scenario



Dashboard 4 · Stack Health – targets, alerts, Prometheus/Alertmanager internals

Prometheus Alerts Status

Filter by rule group state: Filter by rule name or labels

ml_quality_alerts /etc/prometheus/rules/alert_rules.yml

FIRING (4) INACTIVE (3)

FeatureDriftModerate	
FeatureDriftHigh	FIRING (6)
FeatureDriftKS	FIRING (3)
MultipleFeaturesDrifted	FIRING (1)
PredictionDrift	FIRING (1)
PositiveRateAnomaly	
DriftExporterStale	

operational_alerts /etc/prometheus/rules/alert_rules.yml

FIRING (1) INACTIVE (6)

ModelServiceDown	
UpstreamModelUnreachable	
HighLatencyP95	FIRING (1)
HighLatencyP99Critical	
HighErrorRate	
NoTraffic	
TrafficSpike	

Prometheus – alert rules (pending/firing) during the drift scenario

AlertmanagerAlertsSilencesStatusSettingsHelp

New Silence

FilterGroup

Receiver: AllSilencedInhibitedMuted

+

Silence

Custom matcher, e.g. env="production"

Expand all groups

team4-ml-slack

alertname="FeatureDriftHigh"

+

model="CustomerChurnGradientBoosting"

+

severity="warning"

+

6 alerts

team4-ml-slack

alertname="FeatureDriftKS"

+

model="CustomerChurnGradientBoosting"

+

severity="warning"

+

3 alerts

team4-slack

alertname="HighLatencyP95"

+

model="CustomerChurnGradientBoosting"

+

severity="warning"

+

1 alert

team4-critical

alertname="MultipleFeaturesDrifted"

+

model="CustomerChurnGradientBoosting"

+

severity="critical"

+

1 alert

team4-ml-slack

alertname="PredictionDrift"

+

model="CustomerChurnGradientBoosting"

+

severity="warning"

+

1 alert

Alertmanager – grouped alerts and receivers

mlflow3.15.1

GenAImodel training

telco-churn

Overview

Training runs

Observability

Traces

Sessions

Evaluation

Judges

Review

Datasets

Evaluation runs

Prompts & versions

Playground

Prompts

Agent versions

AI GatewayNew

DocsSettings

Experiments > telco-churn >

Overview

UsageQualityTool calls

Filters

Time Unit: Day

Last 7 days

Detect Issues

Traces

0

No data available for the selected time range

Latency

No data available for the selected time range

Errors

0 (Overall error rate: 0.0%)

No data available for the selected time range

Token Usage

0 (0 input, 0 output)

Tokens per Trace

Detect Issues in Your Traces

Use this button to automatically detect quality issues in your traces using AI. This feature helps you identify problems like correctness, latency, and other quality concerns in your agent.

Got it

MLflow AssistantBeta

Ask questions about your experiments, traces, evaluations, and more.

How do I get started with MLflow GenAI?

What is the trend of token usage?

Why did the error rate spike?

Ask a question...

Ollama

MLflow – experiment runs table (compare / filter / chart / CSV)

Fake Slack – 33 notifications

```
16:09:19 /slack
{
  "channel": "#ml-quality",
  "username": "Alertmanager",
  "attachments": [
    {
      "title": "[FIRING:3] FeatureDriftKS \u001d2013 warning",
      "title_link": "http://4d180568cad2:9093/#/alerts?receiver=team4-ml-slack",
      "text": "``KS-test drift on MonthlyCharges (D=0.000)*\nTwo-sample Kolmogorov-\u001d2013Smirnov test rejects 'same distribution' (p<0.01, D=0.2).\nmodel: 'CustomerChurnGradientBoosting' feature: 'MonthlyCharges' since: 16:08:49\nrunbook: https://github.com/AutomationArtist01/MLOPS/blob/main/docs/GUIDEB00K.md#featuredrift\nKS-test drift on TotalCharges (D=0.000)*\nTwo-sample Kolmogorov-\u001d2013Smirnov test rejects 'same distribution' (p<0.01, D=0.2).\nmodel: 'CustomerChurnGradientBoosting' feature: 'TotalCharges' since: 16:08:34\nrunbook: https://github.com/AutomationArtist01/MLOPS/blob/main/docs/GUIDEB00K.md#featuredrift\nKS-test drift on tenure (D=0.000)*\nTwo-sample Kolmogorov-\u001d2013Smirnov test rejects 'same distribution' (p<0.01, D=0.2).\nmodel: 'CustomerChurnGradientBoosting' feature: 'tenure' since: 16:08:04\nrunbook: https://github.com/AutomationArtist01/MLOPS/blob/main/docs/GUIDEB00K.md#featuredrift\n",
      "fallback": "[FIRING:3] FeatureDriftKS CustomerChurnGradientBoosting warning [ml-quality drift-exporter:9105 drift-exporter drift-exporter ml-monitoring team4] | http://4d180568cad2:9093/#/alerts?receiver=team4-ml-slack",
      "callback_id": "",
      "footer": "",
      "color": "danger",
      "mrkdwn_in": [
        "fallback",
        "pretext",
        "text"
      ]
    }
  ]
}
```

```
16:09:04 /slack
{
  "channel": "#ml-quality",
  "username": "Alertmanager",
  "attachments": [
    {
      "title": "[FIRING:1] PredictionDrift \u2013 warning",
      "title_link": "http://4d180568cad2:9093/#/alerts?receiver=team4-ml-slack",
      "text": "\u201c\u201cPrediction distribution shifted (PSI 0.628)\u201c\nThe distribution of model scores differs from the training-time score distribution \u2013 input drift or a serving bug (e.g. wrong feature order)\u201c\nCustomerChurnGradientBoosting feature: \u2013 since: 16:08:49\nrunbook: https://github.com/AutomationArtist01/MLOPS/blob/main/docs/GUIDEBOOK.md#predictiondrift\u201c",
      "fallback": "[FIRING:1] PredictionDrift CustomerChurnGradientBoosting warning (ml-quality drift-exporter:9105 drift-exporter drift-exporter ml-monitoring team4) ]",
      "url": "http://4d180568cad2:9093/#/alerts?receiver=team4-ml-slack",
      "callback_id": "",
      "footer": "",
      "color": "danger",
      "markdown_in": [
        "fallback",
        "pretext",
        "text"
      ]
    }
  ]
}
```

```
16:08:49 /slack
{
  "channel": "#ml-quality",
  "username": "AlertManager",
  "attachments": [
    {
      "title": "[FIRING:2] FeatureDriftKS v2013 warning",
      "text": "https://4d180568cad2:9093/#/alerts?receiver=team4-ml-slack",
      "text": "*KS-test drift on TotalCharges (D=0.000) v2013Smirnov test rejects 'same distribution' (p<0.01, D=0.2).\\nmodel: 'CustomerChurnGradientBoosting' feature: 'TotalCharges' since: 16:08:34\\nrunkbook: https://github.com/AutomationArtist01/MLOPS/blob/main/docs/GUIDEBOOK.md#featuredrift\\nKS-test drift on tenure (D=0.000) v2013Smirnov test rejects 'same distribution' (p<0.01, D=0.2).\\nmodel: 'CustomerChurnGradientBoosting' feature: 'tenure' since: 16:08:04\\nrunkbook: https://github.com/AutomationArtist01/MLOPS/blob/main/docs/GUIDEBOOK.md#featuredrift\\n",
      "callback": "[FIRING:2] FeatureDriftKS v2013 warning\\nCustomerChurnGradientBoosting warning (ml-quality drift-exporter:9105 drift-exporter drift-exporter ml-monitoring team4) | http://4d180568cad2:9093/#/alerts?receiver=team4-ml-slack",
      "callback_id": "",
      "footer": "",
      "color": "danger",
      "mrkdwn_in": [
        "fallback"
      ]
    }
  ]
}
```

Notification sinks – fake Slack (webhook catcher) and Mailhog inbox

 MailHog

 GitHub



50
 1-5 of 5
 <
 >

alerts@team4.local oncall@team4.local	[CRITICAL] MultipleFeaturesDrifted	2 minutes ago	11.4 kB
alerts@team4.local oncall@team4.local	[CRITICAL] HighErrorRate	14 minutes ago	10.83 kB
alerts@team4.local oncall@team4.local	[CRITICAL] MultipleFeaturesDrifted	19 minutes ago	11.43 kB
alerts@team4.local oncall@team4.local	[CRITICAL] HighErrorRate	19 minutes ago	10.8 kB
alerts@team4.local oncall@team4.local	[CRITICAL] MultipleFeaturesDrifted	28 minutes ago	11.4 kB

 Connected

Inbox (5)

 Delete all messages

Jim
 Jim is a chaos monkey.
[Find out more at GitHub.](#)

Enable Jim

Mailhog – critical alert e-mail

3 · Prometheus & PromQL

3.1 Concept

Prometheus (CNCF, originally SoundCloud 2012) is a **time-series database + scraper + query language + alert-rule engine** in one binary. Every N seconds it HTTP-GETs `/metrics` from each configured *target*, parses the text exposition format, and stores samples `(metric name, labels) → (timestamp, value)`. Rules are evaluated over that data; results are graphed (Grafana) or sent to Alertmanager.

Data model: a **time series** is identified by a metric name plus a set of key/value **labels**:

```
mlgw_request_latency_seconds_bucket{model="CustomerChurnGradientBoosting", le="0.1"} 294
```

Labels are the super-power: `sum by (model), {status=~"5.."}, topk(8, drift_psi) ...`

3.2 Alternatives & why Prometheus

Tool	Type	Strengths	Weaknesses vs our needs
Prometheus	pull, dimensional TSDB, PromQL, built-in alerting	de-facto standard for containers/K8s; every language has a client lib; hundreds of exporters; Grafana native; free; single binary; alert rules with <code>for:</code>	not for long-term/multi-tenant storage out of the box (→ Thanos/Cortex/Mimir), no logs/traces
InfluxDB + Telegraf	push, TSDB, Flux/InfluxQL	good for IoT/high-write; SQL-ish	weaker ecosystem for app metrics; alerting via Kapacitor/InfluxDB tasks less mature; v2/v3 churn
Graphite + StatsD	push, hierarchical names	very simple, old and stable	no labels (dot-paths only) → no <code>sum by (model)</code> ; no histograms/quantiles server-side
Datadog / New Relic / Dynatrace	SaaS agents	everything included, APM, ML-ish anomaly detection	cost per host/metric; vendor lock-in; not runnable offline in a classroom
AWS CloudWatch	cloud-native push	zero-ops on AWS	AWS only; slow (1-min granularity), expensive custom metrics
VictoriaMetrics / Thanos / Mimir	Prometheus-compatible	long-term, HA, cheap storage	<i>extensions</i> of Prometheus, not replacements — we'd add them later
OpenTelemetry (OTLP) + Collector	vendor-neutral <i>protocol</i> for metrics/logs/traces	one SDK for all three pillars	still needs a backend — most often Prometheus ; the Collector can <i>export to</i> Prometheus. Complementary, not competing.
ELK / OpenSearch	log search	great for logs	wrong tool for numeric time series & alert math

Why Prometheus for us: the assignment says "Prometheus exporters" — but we would choose it anyway: (1) label-based model lets one stack serve N models; (2) pull model gives *free* health checking (`up`); (3) histograms + `histogram_quantile` give p50/p95/p99 without the app doing statistics; (4) native alert rules with `for:` durations and Alertmanager; (5) `prometheus_client` for Python is 3 lines; (6) runs offline in Compose; (7) it is what the industry (and Kubernetes) uses, so the skill transfers.

3.3 Pull vs push

	Pull (Prometheus)	Push (StatsD, Pushgateway, OTLP, Datadog agent)
Health detection	free: a failed scrape = <code>up == 0</code>	silence is ambiguous: dead or idle?
Config	central (<code>prometheus.yml</code>) — the monitor decides what/how often	every app must know the collector's address
Load control	scraper controls cadence; can't be flooded	bursty clients can overload the collector
Short-lived jobs	awkward (need Pushgateway)	natural
Firewalls	monitor must reach targets	targets must reach monitor

Our services are long-running containers on one network → pull is the natural fit. (The trainer is a one-shot job; if we wanted its metrics we would use the Pushgateway — we didn't need to; MLflow holds its results.)

3.4 Metric types

Type	Meaning	Example here	Typical PromQL
Counter	monotonically increasing total	<code>mlgw_requests_total</code> , <code>prediction_errors_total</code>	<code>rate()</code> , <code>increase()</code>

Type	Meaning	Example here	Typical PromQL
Gauge	value that goes up and down	<code>drift_psi, mlgw_inflight_requests, mlgw_upstream_up</code>	<code>raw value, avg_over_time(), max_over_time()</code>
Histogram	counts in cumulative buckets <code>_bucket{le=...} + _sum + _count</code>	<code>mlgw_request_latency_seconds, mlgw_prediction_score</code>	<code>histogram_quantile(), _sum/_count</code> for the mean
Summary	client-side pre-computed quantiles	(not used — see 3.5)	direct value
Info	constant 1 carrying metadata labels	<code>model_info{name,version,framework}</code>	joins

Naming convention: `<namespace>_<name>_<unit>[_total]` — `mlgw_request_latency_seconds, drift_psi, prediction_requests_total`.
Base units (seconds, not ms; bytes, not MB).

3.5 Histogram vs Summary — and p95 vs average

Summary computes quantiles **inside the application** over a sliding window; you get exact per-instance quantiles but you **cannot aggregate** them: the p95 of two replicas is *not* the average of their p95s. *Histogram* just counts observations per bucket; Prometheus aggregates buckets across instances (`sum by (le)`) and *then* estimates the quantile with `histogram_quantile`. Slight interpolation error inside a bucket, huge win in flexibility (any quantile, any time later, any grouping). Choose buckets around your SLO — ours: 5 ms ... 10 s with 0.5 s (the SLO) as a boundary.

Why **p95/p99** instead of the average? Latency is long-tailed. Ten requests: nine at 20 ms, one at 2 s → average 218 ms ("fine"), p95 ≈ 2 s ("one in twenty users waits 2 s"). Averages hide tail pain; percentiles express user experience. We show both on the "Latency p50/p95/p99/avg" panel — under the *latency* demo scenario the avg line sits far below p99.

3.6 PromQL cookbook — the whiteboard items

Open <http://localhost:9090/query> (or Grafana → Explore). Each query below is copy-paste ready.

Target health

```
up                                # 1 per healthy target, 0 when a scrape fails
up == 0                          # only the dead ones
up{job="model-api"}
count(up == 1) / count(up)      # share of healthy targets
```

Try: `docker compose stop model-api` → `up{job="model-api"}` becomes 0 within one scrape and `ModelServiceDown` fires after 30 s.
`docker compose start model-api` to recover.

Rate of change (counters)

```
rate(mlgw_requests_total[1m])      # per-second rate, averaged over the last 1 min
sum(rate(mlgw_requests_total[1m])) # across all statuses/models
sum by (status) (rate(mlgw_requests_total[5m])) # split by HTTP status
increase(mlgw_requests_total[1h])  # absolute number of requests in the last hour
irate(mlgw_requests_total[1m])     # instantaneous rate (last two samples) – spiky
```

`rate()` handles counter resets (container restart) and is the right function for alerts/graphs; `irate()` is for zooming into spikes; `increase()` for "how many in a window".

Average over time (gauges & derived series)

```
avg_over_time(drift_psi{feature="tenure"}[10m]) # smoothed PSI
avg_over_time(ml[[:]latency_p95[:]5m[1h])      # average of the p95 SLI over an hour
max_over_time(mlgw_inflight_requests[15m])     # peak concurrency
avg(ml[[:]latency_p95[:]5m)                     # average across models *now* (not over time!)
```

`avg()` aggregates *across series at one instant*; `avg_over_time()` aggregates *one series across time*.

Time ranges — 5m · 1h · 1d · 1 month

```
rate(mlgw_requests_total[5m])      # short window: reacts fast, noisier
rate(mlgw_requests_total[1h])      # smooth trend
increase(mlgw_requests_total[1d])   # requests per day
increase(mlgw_requests_total[30d])  # requests per month (needs 30 d of retention – ours is 15 d)
```


Rule of thumb: the range should be $\geq 4 \times$ the scrape interval (5 s scrape $\rightarrow \geq 20$ s; we use 1 m/5 m). Larger windows = smoother, slower to react. Alerts use 5 m; SLO availability uses 30 m; dashboards let you pick. In Grafana, the time-picker (top right) sets *which* period is drawn; the [5m] inside the query sets the *smoothing window* of each point — two different "time ranges".

Graph vs table

In the Prometheus UI every query has **Table** and **Graph** tabs. *Table* runs an *instant query* (one value per series at one timestamp — best for `up`, `drift_psi`, `topk`). *Graph* runs a *range query* (one value per step over the selected period — best for `rate()`, quantiles). Try `sort_desc(drift_psi)` in Table (ranked list) and `drift_psi{feature="tenure"}` in Graph (trend). Grafana mirrors this: the "PSI per feature (now)" bar-gauge and "Drift table" use *Instant* queries; the "PSI over time" panel uses a range query.

Percentiles from a histogram

```

histogram_quantile(0.95, sum by (le) (rate(mlgw_request_latency_seconds_bucket[5m])))
histogram_quantile(0.99, sum by (model, le) (rate(mlgw_request_latency_seconds_bucket[5m])))
sum(rate(mlgw_request_latency_seconds_sum[5m])) / sum(rate(mlgw_request_latency_seconds_count[5m])) # mean

```

Error ratio & availability

```

sum(rate(mlgw_requests_total{status=~"5.."}[5m])) / sum(rate(mlgw_requests_total[5m]))
1 - (sum(rate(mlgw_requests_total{status=~"5.."}[30m])) / sum(rate(mlgw_requests_total[30m])))

```

Gotcha we hit: when *no* 5xx has ever happened, the numerator series doesn't exist and the whole expression is *empty*, not 0 $\rightarrow$ Grafana shows "No data". Fix in the recording rules: `... or (sum by (model)(rate(mlgw_requests_total[5m])) * 0)` zero-fills.

Prediction distribution

```

sum by (le) (increase(mlgw_prediction_score_bucket[5m])) # histogram of scores, last 5 min
sum(rate(mlgw_predictions_total{predicted_class="1"}[5m])) / sum(rate(mlgw_predictions_total[5m])) # positive rate

```

Drift

```

drift_psi > 0.25 # drifted features
topk(5, drift_psi) # worst five
drift_ks_pvalue < 0.05
drift_features_drifted

```

3.7 Recording rules — why

`monitoring/prometheus/rules/recording_rules.yml` pre-computes `ml:latency_p95:5m`, `ml:error_ratio:5m`, ... every 5 s. Benefits: (1) dashboards and alerts share *one* definition of "p95"; (2) heavy `histogram_quantile` over many buckets is evaluated once, not per panel refresh; (3) alert expressions become readable (`ml:latency_p95:5m > 0.5`). Naming convention: `level:metric:operations`.

3.8 Configuration walk-through (`monitoring/prometheus/prometheus.yml`)

- `scrape_interval: 5s` — demo-friendly; production is usually 15–60 s (storage $\propto 1/\text{interval}$).
- `evaluation_interval: 5s` — how often rules and alerts are evaluated.
- `rule_files` — recording + alert rules.
- `alerting.alertmanagers` — where to push alerts.
- `scrape_configs` — one *job* per service; `static_configs` because Compose gives stable DNS names. In Kubernetes this becomes `kubernetes_sd_configs` / `ServiceMonitors`.
- Hot reload after edits: `make reload-prom` (`POST /-/reload`, enabled by `--web.enable-lifecycle`).
- Retention: `--storage.tsdb.retention.time=15d`.

3.9 Troubleshooting

Symptom	Check
target DOWN in <code>http://localhost:9090/targets</code>	container running? <code>docker compose ps</code> ; can Prometheus resolve the name? (<code>docker compose exec prometheus wget -qO- http://gateway:8080/metrics</code>)
query returns nothing	metric name typo? use the metrics explorer (globe icon); label values case-sensitive; counter needs <code>rate()</code> for graphs
<code>histogram_quantile</code> returns NaN	no samples in the window yet, or you forgot <code>sum by (le)</code>

Symptom	Check
rule file rejected	<code>docker run --rm -v \$PWD/prometheus/rules:/r --entrypoint promtool prom/prometheus:v3.5.0 check rules /r/*.yml</code>
stale values	<code>up==1</code> but app not updating? gauges keep last value — look at <code>_created</code> / timestamps, or add a "last update" metric like <code>drift_last_compute_timestamp</code>

3.10 Questions you may be asked

- *Why `rate()` not raw counter?* — counters only go up; the slope is the signal; `rate` also handles resets.
 - *`rate` vs `irate` vs `increase`?* — averaged per-second / last-two-samples per-second / total in window.
 - *Why did you scrape every 5 s?* — demo responsiveness; production 15–30 s. Trade-off: storage & load.
 - *How does Prometheus find targets?* — static list here; service discovery (Kubernetes, Consul, DNS, EC2) in production.
 - *How long is data kept?* — 15 d local; longer via remote-write to Thanos/Mimir/VictoriaMetrics.
 - *Cardinality?* — every unique label combination is a series. We label by `model`, `status`, `feature`, `le` — bounded sets. Never label by `user-id` / `request-id`.
-

4 · Exporters — the four operational signals

4.1 Concept

An **exporter** is anything that serves Prometheus text on `/metrics`. Three ways to get one:

1. **Instrument the app itself** with a client library (`prometheus_client` for Python) — "library style".
2. **Sidecar / proxy** that observes traffic to an un-instrumented app — our *gateway*.
3. **Stand-alone exporter** that computes something and exposes it — our *drift exporter* (like `node_exporter`, `blackbox_exporter`, `mysqld_exporter` in the wider ecosystem).

We use all three so the trade-offs can be shown side by side.

4.2 The signals the assignment asks for — and how each is measured

Signal	Metric (gateway, model-agnostic)	Metric (in-process, model_api)	PromQL
Request rate	<code>mlgw_requests_total{model,status}</code> (Counter)	<code>prediction_requests_total{model,version}</code> + <code>http_requests_total</code> (instrumentator)	<code>sum(rate(...[1m]))</code>
Latency p50/p95/p99	<code>mlgw_request_latency_seconds</code> (Histogram, 11 buckets 5 ms–10 s)	<code>prediction_request_latency_seconds</code> (Histogram)	<code>histogram_quantile(0.95, sum by (le) (rate(..._bucket[5m])))</code>
Error rate	<code>mlgw_requests_total{status=~"5.."} + mlgw_errors_total{reason}</code> (upstream_5xx / client_4xx / timeout / connection)	<code>prediction_errors_total{reason}</code>	ratio of rates
Prediction distribution	<code>mlgw_prediction_score</code> (Histogram, buckets 0.1...1.0) + <code>mlgw_predictions_total{predicted_class}</code> + <code>mlgw_last_score</code>	<code>prediction_probability</code> (Histogram) + <code>predictions_total{predicted_class}</code>	<code>sum by (le) (increase(..._bucket[5m]))</code> , positive-rate ratio
Saturation (bonus)	<code>mlgw_inflight_requests</code> (Gauge)	—	<code>max_over_time</code>
Health (bonus)	<code>mlgw_upstream_up</code> (Gauge, /health probe every 10 s)	<code>model_loaded</code> , <code>model_info</code>	<code>== 0</code>

4.3 The gateway (`services/gateway/gateway.py`) — how it works

- FastAPI + `httpx.AsyncClient`. `POST /predict` → forwards body to `UPSTREAM_URL+PREDICT_PATH`, returns the upstream response byte-for-byte (status code and content-type preserved) so clients see no difference.
- Around the call: `INFLIGHT.inc()/dec()`, `time.perf_counter()` → `LAT.observe()`, `REQS.labels(status).inc()`, error classification (5xx / 4xx / timeout / connection error).
- On 2xx it parses the JSON, reads `PROB_FIELD` (dotted path allowed, e.g. `result.score`) and `LABEL_FIELD` and observes them.
- It then `asyncio.create_task(_send_drift(...))` — the drift feed never blocks the response.
- Background task probes `HEALTH_PATH` every 10 s → `mlgw_upstream_up`.
- `/metrics` is served by `prometheus_client.generate_latest()`.

Configuration is **only env vars** (see `docker-compose.yml` → `gateway.environment`). Overhead measured in the demo: gateway p95 minus in-process p95 ≈ 1–3 ms (see the panel "model_api p95 (in-process) vs p95 gateway").

4.4 In-process instrumentation (`services/model_api/api.py`)

Ten lines with `prometheus_client`:

```
from prometheus_client import Counter, Histogram
REQ = Counter("prediction_requests_total", "", ["model", "version"])
LAT = Histogram("prediction_request_latency_seconds", "", ["model", "version"],
               buckets=(0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10))
PROB = Histogram("prediction_probability", "", ["model", "version"], buckets=[0.1, 1, 1.0])

@app.post("/predict")
def predict(x):
    REQ.labels(MODEL, VER).inc()
    with LAT.labels(MODEL, VER).time():
        p = model.predict_proba(...)[0, 1]
    PROB.labels(MODEL, VER).observe(p)
```

plus `prometheus-fastapi-instrumentator` for generic `http_*` metrics and `/metrics`. Snippets for Flask, plain Python and non-Python are in `docs/wiring-guide.md`.

4.5 Library vs sidecar — trade-offs

	In-process library	Gateway / sidecar
Code changes to the model	yes (every model, every language)	none
Sees	inside the process (model time only)	what the client sees (network + serialisation + queueing)
ML semantics (score, class)	trivial	via JSON field names (<code>PROB_FIELD</code>)
Failure modes visible	exceptions inside handler	also timeouts, connection refused, container down
Extra hop / latency	none	~1–3 ms
Reusable across teams	copy code	change env vars

Recommendation we document: **gateway by default** for reusability; add in-process metrics when you need model-internal detail (feature-fetch time, cache hits, per-stage timing).

4.6 Label design & cardinality

Labels: `model` (bounded), `status` (≈ 5 values), `predicted_class` (2), `reason` (4), `feature` (19), `le` (11). Total series $\approx$ a few hundred per model. Never put unbounded values (customer id, request id, raw score) in labels — that is what histograms are for.

4.7 The bug we found in the sample model — why "prediction distribution" matters

The other team's `api.py` re-fits a `ColumnTransformer` at start-up with transformers ordered (*categorical, numeric*); the model was trained with (*numeric, categorical*). Because a NumPy array is passed to `predict`, scikit-learn cannot check names $\rightarrow$ columns silently scrambled. On the full training CSV: correct order **AUC 0.94, mean score 0.25**; API order **AUC 0.53, mean score 0.09**. Their live Render deployment has this bug.

How our stack would catch it *without any labels*: `mlgw_prediction_score` histogram / `ml:score_mean` / `ml:positive_rate` are far off the training reference $\rightarrow$ **PredictionDrift** and **PositiveRateAnomaly** fire while every operational signal is green. In the demo you can literally see it: `docker-compose.second-model.yml` runs their unmodified image behind a second gateway (`model="churn-original"`); the same customer gets 0.069 there vs 0.087 from the fixed service, and the score histograms differ.

Fix in `services/model_api/api.py`: one line, transformer order matches training. (Please tell the other team.)

4.8 Demo

```
make predict                # one request through the gateway
curl -s localhost:8080/metrics | grep mlgw_request_latency_seconds_bucket
make scenario-latency       # p95 climbs > 500 ms → HighLatencyP95 within ~1 min
make scenario-errors        # 30 % 5xx → HighErrorRate
make scenario-burst         # 40 rps → TrafficSpike
make scenario-bad           # malformed JSON → client_4xx errors (not counted as 5xx)
make scenario-normal
```

4.9 Questions you may be asked

- *Why histogram buckets 5 ms–10 s?* — cover the SLO (0.5 s) with boundaries on both sides; too many buckets = more series; too few = coarse quantiles.
- *Why is gateway p95 higher than in-process p95?* — it includes network, JSON (de)serialisation and uvicorn queueing — the *user's* latency.
- *Would you put the gateway in production?* — yes for small fleets; at scale the same role is played by a service mesh sidecar (Envoy emits identical golden-signal metrics) plus a small ML-aware filter.
- *What about batch models?* — expose metrics from the batch job via Pushgateway or write a small exporter that reads the output table — the drift exporter's `/ingest` also accepts batches.

5 · Drift — PSI, KS and the rolling window

5.1 Concepts

- **Data drift / covariate shift** — the *input* distribution $P(X)$ changes (customers get younger, prices rise). Detectable **without labels** — this is what our exporter measures.
- **Prediction drift** — the distribution of the model's *outputs* changes. Also label-free; a cheap proxy for "something changed" (input drift or a serving bug, §4.7). We measure PSI on the score histogram.
- **Concept drift** — the relationship $P(Y|X)$ changes (same customers, different churn behaviour after a competitor's offer). Only detectable **with ground-truth labels** (accuracy over time), which arrive late for churn. Out of scope for a real-time exporter; §10 explains how an "accuracy exporter" would plug in.
- **Reference / baseline** — the training data profile. **Current window** — the last N live rows.

5.2 Why drift monitoring matters

A model is a function fitted to a distribution; feed it a different distribution and its guarantees vanish silently (200 OK, wrong answer). Drift is the *leading indicator* you can act on before labels arrive: retrain, roll back, alert the data owners.

5.3 The statistics

PSI — Population Stability Index

For binned distributions *expected* (reference) e_i and *actual* (window) a_i :

$$\text{PSI} = \sum_i (a_i - e_i) \cdot \ln(a_i / e_i)$$

- Symmetric-ish KL divergence; works for **numeric (binned) and categorical** features alike.
- Industry rule of thumb (credit-scoring origin): $** < 0.10$ stable · $0.10\text{--}0.25$ moderate shift ·

0.25 significant shift**. We alert `info` at 0.10 and `warning` at 0.25.

- Implementation details that matter: bins from **reference quantiles** (10 bins) so each reference bin holds ~10 % — robust to skew; outer bins open $(-\infty, +\infty)$ so out-of-range values are counted; tiny $\epsilon=1e-4$ added to avoid $\ln(0)$; unseen categories go to an "other" bucket.

KS — two-sample Kolmogorov–Smirnov test

$D = \max_x |F_{\text{ref}}(x) - F_{\text{window}}(x)|$ — the largest gap between the two empirical CDFs; the p-value says how surprising D is if both samples came from one distribution.

- **Numeric only**, non-parametric, no binning choices.
- With big samples the p-value goes to ~0 for tiny, harmless differences → we alert on **$p < 0.01$ AND $D > 0.2$** (effect size), not on p alone.
- Reference sample: 2 000 random training rows stored in the JSON (`values`).

Why both

PSI is interpretable and covers categoricals; KS is a proper statistical test for numerics and needs no bins. Agreement between them raises confidence; on the demo drift both fire for `tenure`, `MonthlyCharges`, `TotalCharges` while only PSI can fire for `Contract`, `PaymentMethod`, `InternetService`.

Alternatives considered

Method	Notes
Jensen–Shannon divergence	bounded 0–1, symmetric; fine alternative to PSI
Wasserstein / Earth-mover	distance in feature units, sensitive to shift magnitude; numeric only
Chi-square test	categorical equivalent of KS
Classifier-based (domain classifier AUC)	strongest multivariate signal; heavier
Multivariate (MMD, PCA-reconstruction)	catches joint drift; harder to explain per feature

We chose **PSI + KS**: the assignment names them, they are the industry defaults, cheap enough for a 15-second loop, and explainable per feature on a bar chart.

5.4 Rolling window design

- `deque(maxlen=WINDOW_SIZE=500)` — the last 500 predictions; $O(1)$ append; memory-bounded.
- `MIN_ROWS=50` — no statistics until enough rows (avoid noisy PSI on 5 samples).
- Recompute every `COMPUTE_INTERVAL=15` s in a background thread; gauges are updated atomically-ish (Prometheus scrapes whatever the last computation produced).
- Why a **count-based** window instead of time-based? Statistical power depends on n , not minutes. Trade-off: at 5 rps 500 rows $\approx$ 100 s of traffic $\rightarrow$ detection latency $\approx$ 2 min in the demo. Larger window = fewer false alarms, slower detection. Tune per model traffic.
- Alternative designs: fixed batches (Evidently reports hourly), exponentially-weighted histograms, windows per segment (per country / per plan) — easy to add as an extra label.

5.5 Reference profile (`services/drift_exporter/build_reference.py`)

```
python services/drift_exporter/build_reference.py --csv data/telco_churn.csv \
  --numeric SeniorCitizen tenure MonthlyCharges TotalCharges \
  --categorical gender Partner ... PaymentMethod \
  --score-csv scores.csv --out services/drift_exporter/reference/telco_reference.json
```

Produces per numeric feature {edges, probs, values}; per categorical {probs}; optional `score` histogram (training-time model scores $\rightarrow$ prediction-drift baseline). Regenerate on every retrain.

5.6 Metrics exposed

Metric	Type	Meaning
<code>drift_psi{model,feature,type}</code>	Gauge	PSI per feature (type = numeric/categorical)
<code>drift_ks_statistic{feature}, drift_ks_pvalue{feature}</code>	Gauge	KS for numeric features
<code>drift_prediction_psi{model}</code>	Gauge	PSI of the score distribution
<code>drift_features_drifted{model}</code>	Gauge	count of features with PSI > threshold
<code>drift_window_rows, drift_rows_ingested_total</code>	Gauge / Counter	window fill & feed health
<code>drift_last_compute_timestamp, drift_compute_seconds</code>	Gauge	liveness of the loop (alert <code>DriftExporterStale</code>)
<code>drift_window_mean, drift_reference_mean</code>	Gauge	intuition panel: live vs training means

5.7 Alerts (see `monitoring/prometheus/rules/alert_rules.yml`)

`FeatureDriftModerate` (info, PSI 0.10–0.25) · `FeatureDriftHigh` (warning, PSI > 0.25, for 1 m) · `FeatureDriftKS` (warning) · `MultipleFeaturesDrifted` (**critical**, ≥ 3 features $\rightarrow$ retrain) · `PredictionDrift` (warning, score PSI > 0.25) · `PositiveRateAnomaly` · `DriftExporterStale`. Inhibition: `FeatureDriftHigh` silences `FeatureDriftModerate` for the same feature.

5.8 Demo

```
make scenario-drift          # loadgen shifts tenure, MonthlyCharges, Contract-Month-to-month, Fiber optic, e-check
# Grafana  $\rightarrow$  "3 · Data Drift": bars turn red within ~2 min; PSI over time crosses 0.25;
# KS D rises; means diverge; Alertmanager shows FeatureDriftHigh (x6), FeatureDriftKS (x3),
# MultipleFeaturesDrifted (critical  $\rightarrow$  #ml-oncall + e-mail), PredictionDrift.
make scenario-normal        # window flushes over the next ~2 min; alerts resolve (send_resolved: true)
```

5.9 Troubleshooting

- `drift_window_rows` stays 0 $\rightarrow$ gateway `DRIFT_URL` wrong or drift service down (`mlgw_drift_events_total{status="error"}` grows).
- PSI huge for a feature at baseline $\rightarrow$ reference built from a different schema/units, or a feature the API renames; check `build_reference.py` args.
- KS p-value 0 always $\rightarrow$ large n ; rely on `D`; that's why the alert requires `D > 0.2`.
- Categorical PSI jumps when a *new* category appears $\rightarrow$ the "other" bucket; expected & informative.

5.10 Questions you may be asked

- *Data vs concept drift?* — $P(X)$ vs $P(Y|X)$; labels needed only for the latter.
 - *PSI thresholds — where from?* — credit-risk practice; not laws. Calibrate per feature; we expose `drift_psi_threshold` so it's visible.
 - *Why not just monitor accuracy?* — labels come months later; drift is real-time.
 - *Why not Evidently / whylogs / NannyML / Arize?* — excellent libraries (Evidently in particular for reports and 100+ tests); the assignment asks for a *custom exporter* and we wanted a small, transparent, real-time loop with per-feature Prometheus metrics. Evidently could replace the math inside `compute()` in a day; the interface (`/ingest`, `/metrics`) would not change.
 - *What happens after an alert?* — runbook: confirm on the dashboard, check for upstream data-pipeline changes, compare segments, decide retrain vs rollback; retraining regenerates the reference.
-

6 · Grafana — dashboards

6.1 Concept

Grafana is a visualisation & alerting front-end that queries data sources (Prometheus here) and renders panels on dashboards. It stores *no* metrics itself. Key concepts: **data source**, **dashboard**, **row**, **panel** (visualisation + queries + field config + thresholds), **variables** (`$model`), **annotations** (events overlaid on graphs), **provisioning** (config-as-code), **unified alerting**.

6.2 Alternatives & why Grafana

Tool	Notes
Grafana	de-facto UI for Prometheus; PromQL editor with autocomplete; 20+ panel types; provisioning; alerting; free/OSS
Prometheus built-in UI	fine for ad-hoc queries (we use it for the PromQL demo), no dashboards/persistence
Kibana / OpenSearch Dashboards	for Elasticsearch data; poor PromQL story
Chronograf	InfluxDB-only
Datadog / New Relic dashboards	SaaS, cost
Apache Superset / Metabase	BI over SQL, not time-series-first
Perses (CNCF)	new dashboards-as-code tool; promising, immature

6.3 How our dashboards are built — dashboards-as-code

`monitoring/grafana/build_dashboards.py` generates four JSON files into `monitoring/grafana/dashboards/`; Grafana loads them via `monitoring/grafana/provisioning/dashboards/dashboards.yml` (folder "ML Monitoring", `allowUiUpdates: true` so you can still edit live in the demo, refreshed every 10 s). The data sources (Prometheus + Alertmanager) are provisioned in `monitoring/grafana/provisioning/datasources/prometheus.yml`.

Why generate JSON instead of clicking? Reproducible (`git diff` shows what changed), reviewable, identical in every environment, and 40+ panels would be painful by hand. Edit the Python → `make dashboards` → Grafana picks it up.

Dashboard	Purpose	Highlights
1 · Model API – Operational	golden signals	stat row (rps, p95, error ratio, availability gauge, upstream up, in-flight); rate by status; p50/p95/p99/avg with SLO threshold line; error rate & reasons; latency heatmap + bucket bars; demo-state row
2 · Predictions	model behaviour	mean score, positive-rate gauge, prediction PSI, predictions/s by class, score histogram, score heatmap over time; in-process vs gateway comparison
3 · Data Drift	PSI/KS	drifted-count, window rows, PSI bar-gauge (colour thresholds), PSI over time with threshold bands, KS D & p, live-vs-reference means, drift <i>table</i> (instant query)
4 · Stack Health	meta-monitoring	targets up, firing/pending alerts, ALERTS table, TSDB stats, Alertmanager notifications, scrape durations, CPU per exporter

All dashboards: `$model` variable, links bar (to each other, Prometheus, Alertmanager, MLflow), "Critical alerts" annotation (`ALERTS{alertstate="firing",severity="critical"}`), auto-refresh 5 s.

6.4 Hands-on: the whiteboard tasks (do these live)

Log in at `http://localhost:3000` (admin/admin → *Skip* password change).

Edit an existing panel — open *1 · Model API*, hover the "p95 latency" stat → menu (:) → *Edit*. Change the query to `m1:latency_p99:5m`, title to "p99 latency", or the threshold from 0.5 to 0.3 (Field → Thresholds). *Apply*, then *Save dashboard* (⌘S / Save icon). Because provisioning allows UI updates it will persist until the JSON file changes again.

Create a new panel — *Edit* → *Add* → *Visualization* → pick Prometheus → query `sum by (predicted_class) (rate(mlgw_predictions_total[1m]))`, visualisation *Time series* (or *Pie chart* with *Instant*), title "Predictions by class", legend `{{predicted_class}}` → *Apply* → *Save*. Good second example: `drift_features_drifted` as a *Stat* with thresholds 0/1/3.

Set alert thresholds — two meanings, show both: 1. *Visual threshold* on a panel: *Edit* → *Field* → *Thresholds* (e.g. base green, 0.1 orange, 0.25 red) and *Threshold style: line+area* on time series (see the "PSI over time" panel). 2. *Grafana-managed alert rule*: *Edit panel* → *Alert* tab → *New alert rule* → condition `m1:latency_p95:5m IS ABOVE 0.5` for 1 m → folder/evaluation group → contact point. Explain that in *our*

design the *authoritative* alerts live in Prometheus rules (§7) so they are versioned and independent of the UI; Grafana alerts are shown as the alternative (Alerting → Alert rules lists both — the Prometheus datasource is configured with `manageAlerts: true`, so Prometheus alert rules appear in Grafana's UI too).

Change time range — top-right picker: *Last 5 minutes / 30 minutes / 1 hour / 24 hours / 30 days*; or drag-select on any graph to zoom; ⌘/Ctrl-Z to zoom out; the ⏪ ⏩ arrows shift the window. Show how `rate(... [1m])` looks spiky at 5 m and smooth at 24 h, and that at 30 d only the stack's lifetime has data.

Change refresh rate — next to the time picker: dropdown *5s/10s/30s/1m/5m/.../Off*. Dashboards default to 5 s; set 30 s to reduce load; *Off* to freeze for screenshots. Also settable in the URL (`&refresh=10s`) and in dashboard *Settings* → *Time options*.

Bonus: *Explore* (compass icon) for ad-hoc PromQL with *Table/Graph* toggle — the Grafana version of the Prometheus "graph vs table" whiteboard item.

6.5 Panel-type choices (design reasoning)

Question	Panel	Why
single current value with context	Stat (with sparkline)	glanceable, colour thresholds
bounded ratio	Gauge	shows distance to limit
ranking many series <i>now</i>	Bar gauge (instant)	PSI per feature
trend	Time series	percentiles, rates
distribution over time	Heatmap	latency & score histograms — colour = count per bucket
exact numbers	Table (instant, merged)	drift table, ALERTS table

6.6 Troubleshooting

- Dashboards missing → `docker compose logs grafana | grep provisioning`; JSON must be valid; uid must be unique.
- "No data" → run the query in Explore; check the `$model` variable (set *All*); check the time range.
- Panel says *Instant* but shows a line → the query has `instant: true` only in Table format; toggle *Query options* → *Type*.
- Login loop → cookies from another Grafana; use a private window.

6.7 Questions you may be asked

- *Grafana vs Prometheus UI?* — Prometheus UI = ad-hoc queries; Grafana = persistent dashboards, variables, alert UI, many data sources.
 - *Why provision instead of clicking?* — reproducibility, code review, disaster recovery.
 - *Grafana alerts vs Prometheus alerts?* — §7.2.
 - *Why a heatmap?* — a histogram has a shape; a time series of one quantile hides it.
-

7 · Alerting — Prometheus rules + Alertmanager

7.1 Concept

Two separate components, deliberately:

- **Prometheus** evaluates *alerting rules* (PromQL + `for:` duration) every `evaluation_interval` and keeps the state *pending* → *firing* → *resolved*. It pushes firing alerts to Alertmanager (and exposes them as the `ALERTS` metric — used on the Stack-Health dashboard).
- **Alertmanager** turns raw alerts into *notifications*: **grouping** (one message for 6 drifted features instead of 6), **de-duplication**, **routing** by labels (severity, category) to **receivers** (Slack, e-mail, PagerDuty, webhook...), **inhibition** (suppress latency alerts while the service is down), **silences** (maintenance windows), **repeat/resolve** notifications.

7.2 Alternatives & why

Option	Notes
Prometheus rules → Alertmanager (chosen)	rules are versioned YAML next to the metrics; independent of any UI; battle-tested grouping/inhibition/silences; the assignment names it
Grafana-managed alerting	convenient (click on a panel), multi-datasource; but rules live in Grafana's DB unless provisioned; Grafana even embeds an Alertmanager. Fine for small teams; we show it as the alternative in §6.4
PagerDuty / Opsgenie / VictorOps	<i>on-call</i> layers (escalation, schedules) that sit behind Alertmanager as receivers
Nagios / Zabbix / Icinga	host-centric checks; not label/PromQL based
Cloud (CloudWatch alarms, Azure Monitor)	vendor-bound
Datadog monitors	SaaS

7.3 Anatomy of an alert rule (`monitoring/prometheus/rules/alert_rules.yml`)

```
- alert: HighLatencyP95
  expr: ml:latency_p95:5m > 0.5           # PromQL; uses a recording rule (single definition of p95)
  for: 1m                                # must be true continuously for 1 min → avoids flapping
  labels: {severity: warning, team: team4, category: operational} # routing keys
  annotations:
    summary: "p95 latency {{ $value | humanizeDuration }} > 500ms for {{ $labels.model }}"
    description: "..."
    runbook_url: "...#highlatencyp95"      # what to do – see §12
```

Our rule set (14 rules):

Group	Alert	Condition	Severity
operational	ModelServiceDown	up == 0 for 30 s (model-api, gateway, drift-exporter)	critical
	UpstreamModelUnreachable	mlgw_upstream_up == 0 30 s	critical
	HighLatencyP95	p95 > 0.5 s for 1 m	warning
	HighLatencyP99Critical	p99 > 2 s for 1 m	critical
	HighErrorRate	5xx ratio > 5 % (and traffic > 0.5 rps) for 1 m	critical
	NoTraffic	< 0.05 rps for 3 m after having had traffic in the last 30 m	warning
	TrafficSpike	> 25 rps for 1 m	warning
ml-quality	FeatureDriftModerate	0.10 < PSI ≤ 0.25 for 2 m	info
	FeatureDriftHigh	PSI > 0.25 for 1 m	warning
	FeatureDriftKS	p < 0.01 and D > 0.2 for 2 m	warning
	MultipleFeaturesDrifted	≥ 3 features > 0.25 for 1 m	critical
	PredictionDrift	score PSI > 0.25 for 2 m	warning
	PositiveRateAnomaly	positive rate outside [5 %, 60 %] for 3 m	warning
	DriftExporterStale	no recompute for 120 s	warning

Design notes: thresholds are SLO-derived, not arbitrary (0.5 s = our latency SLO, 5 % error budget burn, PSI 0.25 = industry "significant"); and `ml:request_rate:1m > 0.5` guards ratio alerts against noise at near-zero traffic; every alert has a `runbook_url`; `for:` is short for the demo (1–3 m) — production would use 5–15 m.

7.4 Alertmanager configuration (`monitoring/alertmanager/alertmanager.yml`)

```
route (default: team4-email, group_by [alertname, model, severity], group_wait 10s, group_interval 30s, repeat 2h)
├── severity=critical      → team4-critical (Slack #ml-oncall + e-mail)
├── category=ml-quality    → team4-ml-slack (Slack #ml-quality)
├── severity=warning       → team4-slack   (Slack #ml-alerts)
└── severity=info          → team4-email
inhibit_rules:
  - ModelServiceDown = mute everything else for the team
  - HighLatencyP99Critical = mute HighLatencyP95 (same model)
  - FeatureDriftHigh = mute FeatureDriftModerate (same model+feature)
```

- **Slack** — `slack_configs.api_url` = incoming-webhook URL, custom title/text templates in `monitoring/alertmanager/templates/slack.tpl` (summary, description, model/feature, since, runbook link).
- **E-mail** — `smtp_smarthost`, `smtp_from`, `email_configs.to`; subject templated with status/name.
- **Env substitution** — Alertmanager has none, so `monitoring/alertmanager/entrypoint.sh` `sed s ${SLACK_WEBHOOK_URL}, ${SMTP_*}, ${ALERT_EMAIL_TO}` from the container env at start-up. Defaults route to the demo sinks; a real `.env` (see `.env.example`) switches to a real Slack workspace / SMTP without touching YAML.
- **Demo sinks** — **Mailhog** (SMTP :1025, inbox UI :8025) and the **webhook-catcher** (:8091), a 30-line FastAPI app that stores the JSON Slack would have received and shows it on a page.
- `send_resolved: true` everywhere → you also see the green "RESOLVED" messages after `make scenario-normal`.

Verified in the demo run: 12 Slack notifications (correct channels per route) and the critical e-mail for `MultipleFeaturesDrifted`; `HighErrorRate` at 16 % error ratio.

7.5 Grouping, inhibition, silences — why they matter

Without grouping, the drift scenario would send **~12 separate messages**; with `group_by [alertname, model, severity]` Slack gets one "[FIRING:6] FeatureDriftHigh". Without inhibition, a dead model would page for latency, errors *and* drift; with it, one "ModelServiceDown". Silences (Alertmanager UI → *New silence*, matcher `alertname=TrafficSpike`) are how you do maintenance without deleting rules — show it live.

7.6 Operational vs ML-quality alerts — routing philosophy

Operational alerts wake the on-call **SRE** (Slack #ml-oncall / pager). ML-quality alerts go to the **data-science** channel (#ml-quality) — nobody should be paged at 3 a.m. for PSI 0.26, but a data scientist should look in the morning; only `MultipleFeaturesDrifted` (model probably invalid) is critical. The `category` label carries this distinction through the whole pipeline.

7.7 Demo

```
make scenario-latency      # → HighLatencyP95 (pending ~1 m, then firing) → Slack #ml-alerts
make scenario-errors       # → HighErrorRate → #ml-oncall + e-mail (critical)
make scenario-drift        # → FeatureDrift*, MultipleFeaturesDrifted (critical), PredictionDrift
docker compose stop model-api # → UpstreamModelUnreachable + ModelServiceDown; inhibition mutes the rest
docker compose start model-api
make scenario-normal
python3 tests/alert_status.py # what fired, where it was routed, what Slack/e-mail received
```

Watch: Prometheus → *Alerts* (pending → firing), Alertmanager UI (grouped, receivers), Mailhog, fake-Slack page, and Grafana 4 · *Stack Health*.

7.8 Troubleshooting

- Alert fires in Prometheus but nothing in Alertmanager → `prometheus.yml` `alerting.alertmanagers` target; `docker compose logs alertmanager`.
- Nothing in Slack/Mailhog → check *Alertmanager* → *Status* shows your receivers; `entrypoint.sh` substitution (`docker compose exec alertmanager cat /tmp/alertmanager.yml`); `alertmanager_notifications_failed_total` on the Stack dashboard.
- Alert never leaves *pending* → the condition is not continuously true for the `for:` window.
- Too many messages → check `group_by`, `group_interval`, `repeat_interval`.

7.9 Questions you may be asked

- *Why `for:`?* — one bad scrape must not page anyone; alerts on state, not on samples.
 - *Pending vs firing?* — condition true but `for:` not yet elapsed vs true for the whole window.
 - *Why not alert on the average latency?* — §3.5.
 - *What is an inhibition rule?* — "if A is firing, do not notify B" (root cause vs symptoms).
 - *How would you page a human?* — add a PagerDuty/Opsgenie receiver on the critical route.
 - *Where do runbooks live?* — annotation `runbook_url` → §12 of this book.
-

8 · MLflow — experiment tracking & model registry

8.1 Concept

Experiment tracking: every training *run* logs parameters, metrics, artifacts (model file, plots), tags, code version → runs become comparable and reproducible. **Model registry:** a versioned catalogue of models (`telco-churn-classifier v1, v2 ...`) with aliases/stages (`@champion`, `@challenger`) so serving code can load "the champion" instead of a file path. MLflow also defines a **model packaging format** (`MLmodel` + flavors) — that is what the other team's `model/` folder is (`mlflow.sklearn flavor`, `skops serialisation`).

Where it sits in the stack: *training-time* observability. Prometheus/Grafana watch the model in production; MLflow remembers how it was made and which one is in production. The registry version is also what the `model_info{version}` metric should be set from.

8.2 Alternatives & why MLflow

Tool	Notes
MLflow (chosen)	open source, self-hostable, language-agnostic, tracking + registry + packaging + serving; the other team already used it, so their artifact loads directly
Weights & Biases	best-in-class UI, sweeps; SaaS (self-host is enterprise)
Neptune, Comet	SaaS trackers, similar to W&B
DVC / DVC Experiments	git-centric data & pipeline versioning; less UI
SageMaker / Vertex / Azure ML Experiments	cloud-bound
ClearML, Aim, TensorBoard	ClearML full MLOps; Aim/TensorBoard mostly visualisation

8.3 What we run

- `mlflow service: mlflow server (v3.15.1)` with SQLite backend + local artifact store in a volume, `--serve-artifacts`, `--allowed-hosts` (MLflow 3 rejects unknown Host headers), `--expose-prometheus` (MLflow's own request metrics are scraped by Prometheus — meta-monitoring). UI: **`http://localhost:5001`**.
- `trainer` (profile `train`, make `train`): trains the churn task on the same CSV with 22 configs (12 GBM, 6 RandomForest, 4 LogReg), logs params/metrics/model/signature per run, registers the best by ROC-AUC as `telco-churn-classifier` with alias `@champion`, and exports all runs to `artifacts/mlflow_runs.csv`. Best in our run: `gbm n_estimators=200 lr=0.05 depth=2`, AUC 0.8455.

8.4 Hands-on: the whiteboard tasks (do these live at `http://localhost:5001`)

Compare runs — Experiments → *telco-churn* → tick 3–5 runs → **Compare**. You get a parallel coordinates plot (params → metric), a scatter, and a side-by-side table of params/metrics.

Filter runs by hyper-parameters / metrics — the search box accepts SQL-like filters: `params.model_family = 'gbm'`, `metrics.roc_auc > 0.84`, `params.max_depth = '2'` and `metrics.f1_score > 0.55`, `tags.team = 'team4'`. Sort by clicking a metric column. Explain the API twin: `mlflow.search_runs(filter_string="metrics.roc_auc > 0.84")`.

Visualise hyper-parameter effects — in the experiment table click *Chart view* (or in Compare): scatter *learning_rate* (x) vs *roc_auc* (y), colour by *model_family*; parallel coordinates *n_estimators*, *learning_rate*, *max_depth* → *roc_auc*. Story: depth 2 beats depth 4 (less over-fit), *lr* 0.05 with 200 trees ≈ *lr* 0.1 with 100.

Inspect the saved / best model — sort by *roc_auc* desc → open the top run → *Artifacts* → *model*: `MLmodel` (flavors, signature with input schema, python env), `model.pkl`, `requirements.txt`, `input_example.json`. Show *Parameters*, *Metrics*, *Tags*, source and duration.

Register a model — from that run's artifact view: *Register model* → new name or existing `telco-churn-classifier` → *Register*. Then *Models* tab → version list → set alias (`champion` / `challenger`), add tags/description. Explain loading by alias: `mlflow.sklearn.load_model("models:/telco-churn-classifier@champion")`.

Export results as CSV for all runs — experiment table → the download icon (*Download CSV*) → all visible runs with params & metrics. Also already produced by the trainer: `artifacts/mlflow_runs.csv` via `mlflow.search_runs(...).to_csv(...)`.

8.5 How MLflow relates to the monitoring stack

- Version label: the model API reports `model_info{version}`; when a new registry version is promoted, the label changes and every dashboard/alert can be split "before vs after".

- Drift reference: retraining = new training data → regenerate `telco_reference.json` (\$5.5) and restart the drift exporter (or mount the new file).
- Closing the loop (future): `MultipleFeaturesDrifted` → trigger `make train` → compare in MLflow → promote → redeploy. The pieces exist; the automation is a webhook away.

8.6 Troubleshooting

- Invalid Host header (403) → add the host to `--allowed-hosts` in `services/mlflow/Dockerfile`.
- Port 5000 busy on macOS (AirPlay Receiver) → we map 5001.
- Trainer can't reach server → it uses `http://mlflow:5000` inside the network, `localhost:5001` outside.
- Registry empty → run `make train`.

8.7 Questions you may be asked

- *Tracking vs registry?* — runs/history vs promoted, versioned models.
 - *Why register?* — decouple serving from file paths; approvals; rollback = change alias.
 - *What is the MLmodel file?* — packaging manifest: flavors (sklearn, pyfunc), signature, env pins.
 - *Why did the other team's model need `sklearn==1.9.0` and `skops`?* — the artifact pins them; skops is the safe (non-pickle) sklearn serialisation MLflow 3 uses.
 - *Optuna vs our grid?* — they used Optuna (35 trials, Bayesian); we log a small grid because the goal here is comparable, demonstrable runs — the tracking is identical either way.
-

9 · Docker Compose — orchestration

9.1 Concept

Docker Compose describes a multi-container application in one YAML: services (image or build context), environment, ports, volumes, dependencies, restart policy, profiles. `docker compose up -d --build` builds and starts all 10 services on one private network where every service reaches the others by name (`http://prometheus:9090`).

9.2 Compose vs Kubernetes (and others)

	Docker Compose	Kubernetes	bare processes / systemd
Scope	one host, dev/demo/small prod	cluster, autoscaling, self-healing, rolling deploys	ad-hoc
Learning curve	minutes	weeks	—
Monitoring stack	this repo	kube-prometheus-stack Helm chart (Prometheus Operator, ServiceMonitors, Grafana, Alertmanager pre-wired)	manual
Service discovery	DNS by service name	DNS + labels + Operator CRDs	none
Fit for a classroom demo on a laptop	✓	heavy (kind/minikube)	fragile

Why Compose: the assignment asks for it; one command reproduces the whole stack on any laptop; the concepts (scrape targets, rules, provisioning) transfer 1:1 to Kubernetes — the `prometheus.yml` static targets would become ServiceMonitors, the dashboards ConfigMaps, the Alertmanager config a Secret. We note that path in §14.

9.3 The services (`docker-compose.yml`)

Service	Image	Role	Persistent volume
model-api	build <code>./services/model_api</code>	other team's churn model + metrics + <code>/chaos</code>	—
gateway	build <code>./services/gateway</code>	model-agnostic exporter/proxy	—
drift-exporter	build <code>./services/drift_exporter</code>	PSI/KS on rolling window	—
load-generator	build <code>./services/load_generator</code>	traffic + demo scenarios	—
prometheus	<code>prom/prometheus:v3.5.0</code>	TSDB + rules	<code>prometheus_data</code> (15 d retention)
alertmanager	<code>prom/alertmanager:v0.28.1</code> (custom entrypoint)	routing	<code>alertmanager_data</code> (silences, notification log)
grafana	<code>grafana/grafana:12.1.0</code>	dashboards	<code>grafana_data</code>
mailhog	<code>mailhog/mailhog</code>	fake SMTP + inbox	—
webhook-catcher	build <code>./services/webhook_catcher</code>	fake Slack	—
mlflow	build <code>./services/mlflow</code>	tracking + registry	<code>mlflow_data</code>
trainer	build <code>./services/trainer</code> (profile <code>train</code>)	one-shot training	writes <code>./artifacts</code>

Design details worth pointing out: * `restart: unless-stopped` everywhere (via a YAML anchor) — a crashed exporter comes back; Prometheus shows the gap. * Config is **bind-mounted read-only** (`prometheus.yml`, rules, alertmanager, Grafana provisioning and dashboards) → edit on the host, `make reload-prom` / Grafana auto-reload; no image rebuild. * Data lives in **named volumes** → `make down` keeps history, `make nuke` wipes it. * `healthcheck` on model-api; `depends_on` for start order. * **Profiles**: trainer only runs with `--profile train` / `docker compose run --rm trainer`. * Everything pinned to versions (reproducible on the demo day). * `docker-compose.second-model.yml` is an **overlay** (`-f a.yml -f b.yml`) that adds a second model and its gateway — the pattern for wiring more models without touching the base file. * Ports: 5000 → 5001 for MLflow because macOS AirPlay occupies 5000.

9.4 Day-to-day commands (Makefile)

```
make up / down / nuke / logs / ps / status / smoke
make train                # MLflow runs
make dashboards           # regenerate Grafana JSON
make reload-prom          # after editing rules/prometheus.yml
make scenario-*           # normal | drift | latency | errors | burst | bad
make predict              # one request through the gateway
```

9.5 Resource footprint

Measured on an M-series Mac, Docker Desktop 8 GB: ~1.2 GB RAM for the whole stack, < 5 % CPU idle, ~15 % under the burst scenario. Prometheus ingests ≈ 400 samples/s at 5 s scrape (see Stack dashboard).

9.6 Troubleshooting

- `port` is already allocated → something on the host uses it; change the *left* side of `ports:` .
- Build slow the first time (sklearn/mlflow wheels) → later builds are cached.
- Apple-silicon: all images used are multi-arch; `node_exporter` / `cadvisor` were left out on purpose because they misbehave in Docker Desktop VMs (add them on Linux hosts).
- `docker compose config` validates the YAML.

9.7 Questions you may be asked

- *Why not Kubernetes?* — scope/complexity vs benefit for a single-host demo; concepts identical.
 - *How would this scale?* — Prometheus federation/remote-write, several gateways per model behind a load balancer (metrics aggregate by `model` label), Alertmanager cluster, Grafana with a DB.
 - *What persists across restarts?* — TSDB, Grafana edits, Alertmanager silences, MLflow runs (volumes).
-

10 · Wiring the stack to a new model (the reuse guide)

Goal of the assignment: *"documentation on wiring this stack to any new model service"*. There are three levels, from zero code changes to full integration. Time estimates are for someone who has read this chapter.

10.1 Level 0 — put the gateway in front (no code changes) · ~10 min

Requirements: your model answers HTTP POST <path> with a JSON body and returns JSON that contains a score and/or a class; optionally a health endpoint.

1. **Add your service and a gateway** to Compose (copy from `docker-compose.second-model.yml`):

```
yml my-model: image: registry/my-model:1.2.0 # or build: ./my-model gateway-my-model: build: ./services/gateway environment: UPSTREAM_URL: http://my-model:9000 # where the model listens PREDICT_PATH: /v1/predict # its predict route HEALTH_PATH: /healthz # any 200 route (default /health) MODEL_NAME: fraud-xgb # becomes the `model` label everywhere PROB_FIELD: result.score # dotted path into the response JSON (optional) LABEL_FIELD: result.label # optional DRIFT_URL: http://drift-my-model:9105 # or "" to disable drift ports: ["8082:8080"]
```
2. **Tell Prometheus** to scrape it — append to `monitoring/prometheus/prometheus.yml`:

```
yml - job_name: gateway-my-model static_configs: [{targets: ["gateway-my-model:8080"], labels: {service: gateway-my-model}}]
```

 then make `reload-prom`.
3. **Send traffic to the gateway** (`:8082/predict`) instead of the model directly.
4. Done: the *1 · Model API* and *2 · Predictions* dashboards show the new model in the `$model` dropdown; all operational alerts apply automatically (they are written against `mlgw_*` and the `ml:*` recording rules, grouped by `model`).

Response shapes that work out of the box: `{"churn_probability": 0.42, "prediction": 1}`, `{"result": {"score": 0.42, "label": "fraud"}}` (dotted paths), lists are not auto-handled — for batch endpoints see 10.4.

10.2 Level 1 — add drift for the new model · ~30 min

1. Build a reference profile from the model's **training data**:

```
bash python services/drift_exporter/build_reference.py --csv train.csv \ --numeric age amount tenure --categorical country channel plan \ --score-csv train_scores.csv --model fraud-xgb --out services/drift_exporter/reference/fraud_reference.json (--score-csv: one column of the model's scores on the training set; enables prediction drift.)
```
2. Run a drift exporter instance for it: `yml drift-my-model: build: ./services/drift_exporter environment: {REFERENCE_PATH: reference/fraud_reference.json, WINDOW_SIZE: "1000", PSI_ALERT: "0.25"}` and a scrape job `drift-my-model`. Feature names in the request JSON must match the CSV columns (the exporter ignores unknown keys and skips missing ones).
3. The *3 · Data Drift* dashboard and all `drift_*` alerts work per `model` label.

Tuning: `WINDOW_SIZE` $\approx 10 \times (\text{number of PSI bins}) \times (\text{categories})$ is a good floor; `MIN_ROWS` 50–100; `COMPUTE_INTERVAL` 15–60 s.

10.3 Level 2 — in-process metrics (optional, richer) · ~1 h

If you own the model code and want internals (feature-fetch time, cache hits, per-stage latency):

FastAPI / Flask (Python)

```
from prometheus_client import Counter, Histogram, Gauge, make_asgi_app # pip install prometheus-client
REQ = Counter("prediction_requests_total", "", ["model", "version"])
ERR = Counter("prediction_errors_total", "", ["model", "version", "reason"])
LAT = Histogram("prediction_request_latency_seconds", "", ["model", "version"],
               buckets=(0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10))
SCORE = Histogram("prediction_probability", "", ["model", "version"], buckets=[i/10 for i in range(1, 11)])
app.mount("/metrics", make_asgi_app()) # FastAPI/Starlette; Flask: use prometheus_flask_exporter
```

Then wrap the predict handler as in `services/model_api/api.py`. Add a scrape job for the service itself and you get the *"in-process vs gateway"* comparison panels for free (rename metrics or add labels if several models share names).

Other languages: official clients exist for Go, Java, .NET, Node, Ruby, Rust — same metric names.

Non-HTTP / batch models: write a tiny exporter that reads the output table and exposes counts, score histograms and last-run timestamps (pattern = `drift_exporter.py` without `/ingest`), or push per-run summaries to a Pushgateway.

10.4 Adapting the gateway to unusual APIs

`services/gateway/gateway.py` is ~150 lines. Typical 5-line changes: * batch responses (`[{"score":...}, ...]`) → iterate and observe each; * score in a header or as plain text → parse accordingly; * gRPC models → wrap with a small HTTP shim or instrument in-process (Level 2).

10.5 Alerts and dashboards for the new model

- Nothing to do for operational + drift alerts (label-driven).
- Model-specific SLOs (e.g. $p_{95} < 100$ ms for fraud): copy `HighLatencyP95`, add `{model="fraud-xgb"}` to `expr`, adjust threshold, `make reload-prom`.
- Model-specific dashboard: duplicate a dashboard in Grafana → set variable default; or add a section in `monitoring/grafana/build_dashboards.py` and `make dashboards`.
- Notifications: give the model's team its own route in `alertmanager.yml` matching `model="fraud-xgb"` (or a `team` label added in the alert rule).

10.6 Checklist

- [] gateway env: `UPSTREAM_URL`, `PREDICT_PATH`, `MODEL_NAME`, `PROB_FIELD / LABEL_FIELD`
 - [] Prometheus scrape job(s) added, `make reload-prom`, target UP
 - [] traffic goes through the gateway
 - [] reference JSON built from training data; drift exporter running; window fills
 - [] `$model` shows the new model on dashboards
 - [] test alerts: stop the model → `UpstreamModelUnreachable`; replay shifted data → `FeatureDriftHigh`
 - [] Alertmanager route/receiver for the owning team
 - [] `model_info` version matches the MLflow registry version
-

11 · Demo script (≈ 20 minutes)

Preparation (10 min before): `make up` → wait ~60 s → `make train` (≈ 2 min) → `python3 tests/smoke_test.py` (all green) → open tabs: Grafana (3000), Prometheus (9090), Alertmanager (9093), MLflow (5001), Mailhog (8025), fake Slack (8091), Model API docs (8000/docs), Load generator (8090/docs). Set Grafana time range *Last 15 minutes*, refresh 5 s. Keep a terminal for `make ...`.

#	Time	What you do	What you say (key points)
1	1'	Show the architecture picture (§2.1)	two families of signals; gateway = model-agnostic; pull model; one command
2	1'	<code>make status</code> · Prometheus → <i>Status</i> → <i>Targets</i>	target health : 8 jobs UP, scrape every 5 s; up metric
3	2'	Prometheus <i>Query</i> : up (Table) · <code>rate(mlgw_requests_total[1m])</code> (Graph) · <code>avg_over_time(ml:latency_p95:5m[10m])</code> · switch [5m] → [1h], time-picker 5m/1h/1d	rate of change, average over time, time ranges, graph vs table (instant vs range query)
4	1'	<code>histogram_quantile(0.95, sum by (le) (rate(mlgw_request_latency_seconds_bucket[5m])))</code>	histogram vs summary; why p95 not average
5	2'	Grafana 1 · <i>Model API</i> : walk the golden-signal row; hover the p95 panel; show recording rules <code>ml:*</code>	SLI/SLO; dashboards-as-code
6	2'	<code>make scenario-latency</code> → p95 turns red; Prometheus <i>Alerts</i> → pending → firing; fake Slack shows the message	for : duration; routing by severity
7	1'	Grafana whiteboard: edit the p95 panel (threshold 0.3), create a panel (<code>drift_features_drifted</code> stat), change time range , change refresh to 30 s, show Alert tab thresholds	say why authoritative alerts live in Prometheus
8	3'	<code>make scenario-drift</code> → Grafana 3 · <i>Data Drift</i> : bars go red, PSI over time crosses 0.25, KS D up, means diverge; drift table (instant)	PSI formula & thresholds; KS; rolling window (500 rows → ~2 min); data vs concept drift
9	1'	Alertmanager UI: grouped <code>FeatureDriftHigh</code> (6), <code>MultipleFeaturesDrifted</code> critical → Mailhog e-mail + #ml-oncall; show inhibition of <i>Moderate</i>	grouping, inhibition, routes; ops vs ML-quality receivers
10	1'	Grafana 2 · <i>Predictions</i> : score histogram/heatmap moved; positive-rate gauge; <code>PredictionDrift</code>	prediction distribution; the column-order bug story (§4.7); second gateway on the original image
11	1'	<code>make scenario-errors</code> (optional) → <code>HighErrorRate</code> ; <code>make scenario-normal</code> → RESOLVED messages	error budget; <code>send_resolved</code>
12	3'	MLflow: compare runs → filter <code>metrics.roc_auc > 0.84</code> → chart lr vs AUC → open best run → artifacts/MLmodel → <i>Models</i> : <code>telco-churn-classifier@champion</code> → <i>Download CSV</i>	tracking vs registry; version label ↔ Prometheus
13	1'	<code>docker-compose.second-model.yml</code> + docs §10: wiring a new model = env vars + one scrape job	reusability claim, proven live
14	—	Q&A — §13	

Backup if something breaks: `python3 tests/alert_status.py` prints the state of everything; `make restart`; screenshots are in `docs/screenshots/`.

12 · Runbooks & troubleshooting

Every alert's `runbook_url` points here. A runbook = *what it means* · *first checks* · *actions*.

ModelServiceDown

Prometheus cannot scrape the target for 30 s. — `docker compose ps`; `docker compose logs <svc> --tail=100`; `docker compose restart <svc>`. If model-api crashed at start-up: model artifact missing or dependency mismatch (sklearn/skops pins). Inhibition mutes latency/error/drift alerts meanwhile.

UpstreamModelUnreachable

Gateway's /health probe fails. — model container down (see above) or `UPSTREAM_URL` / `HEALTH_PATH` misconfigured (`docker compose exec gateway env | grep UPSTREAM`).

HighLatencyP95 / HighLatencyP99Critical

p95 > 500 ms / p99 > 2 s over 5 min. — Is traffic up (`ml:request_rate:1m`)? Is CPU saturated (Stack dashboard, `process_cpu_seconds_total`)? Model-side latency (in-process p95) vs gateway p95: if only the gateway is slow → network/serialisation; if both → model. Demo cause: `/chaos latency (chaos_injected_latency_ms panel)` → `make scenario-normal`. Real causes: bigger batches, cold starts, contention → scale replicas, cache, smaller model.

HighErrorRate

> 5 % 5xx over 5 min. — `mlgw_errors_total{reason}` tells you upstream_5xx vs timeout vs connection. Check model logs for exceptions (schema change upstream? new category? nulls?). Client 4xx are *not* counted here (see `client_4xx reason`) — a surge of 4xx means callers changed.

NoTraffic

< 0.05 rps for 3 min. — callers down, load balancer, or the load-generator stopped (`docker compose ps load-generator`). Not a model problem — but a business one.

TrafficSpike

> 25 rps. — expected campaign? autoscale; watch latency & saturation panels.

FeatureDrift (Moderate/High/KS)

A feature's live distribution differs from training. — Which feature (`drift_psi` bar gauge)? Numeric: compare `drift_window_mean` vs `drift_reference_mean` ; categorical: new category? Upstream ETL change / unit change (cents vs dollars) / seasonal effect / genuine population change. Actions: confirm with the data owner; if genuine → schedule retraining, then rebuild the reference (§5.5); if a bug upstream → fix and reset the window (`curl -X POST localhost:9105/reset`).

MultipleFeaturesDrifted (critical)

≥ 3 features > 0.25. — model likely operating outside its training distribution. Actions: compare prediction distribution & positive rate; consider switching to a fallback/rules or the previous model (`@champion` alias rollback in MLflow); start retraining; inform stakeholders.

PredictionDrift / PositiveRateAnomaly

Score distribution or positive rate far from training. — If features did **not** drift but predictions did → **serving bug** (feature order, scaling, wrong model version — see §4.7) or a deploy; check `model_info{version}` change annotations. If features drifted too → see above.

DriftExporterStale

No recompute for 2 min while the window has rows. — thread died; `docker compose logs drift-exporter` ; restart. Root cause usually a malformed row (non-numeric in a numeric field).

General troubleshooting

Problem	Fix
<code>make up</code> fails on a port	change the host port in <code>docker-compose.yml</code> (5000→5001 already done for MLflow)

Problem	Fix
Docker daemon not running	start Docker Desktop; <code>docker info</code>
Grafana shows <i>No data everywhere</i>	Prometheus target down? datasource URL <code>http://prometheus:9090</code> ? time range?
Alerts fire but no Slack/e-mail	Alertmanager <i>Status</i> page shows the rendered config; <code>docker compose logs alertmanager</code> ; <code>alertmanager_notifications_failed_total</code>
Drift never computes	<code>drift_window_rows < MIN_ROWS</code> ; gateway <code>DRIFT_URL</code> ; <code>mlgw_drift_events_total{status="error"}</code>
Trainer 403 <i>Invalid Host header</i>	MLflow 3 <code>--allowed-hosts</code> (already set)
Model API fails to start	sklearn/mlflow/skops versions must match the artifact (<code>model/requirements.txt</code>)
Everything is slow on a laptop	lower RPS env of load-generator, raise <code>scrape_interval</code> to 15 s
Reset the whole demo	<code>make nuke</code> && <code>make up</code> && <code>make train</code>

13 · Questions the examiner may ask — short answers

Tools & choices

- *Why Prometheus?* — label-based TSDB + scraper + rule engine in one; pull gives free health checks; histograms → server-side percentiles; alerting with `for:`; huge ecosystem; runs offline; industry standard (Kubernetes). Alternatives (Influx, Graphite, Datadog, CloudWatch, OTel) in §3.2.
- *Why Grafana?* — the standard UI for Prometheus, provisioning-as-code, variables, alert UI; alternatives §6.2.
- *Why Alertmanager and not Grafana alerts?* — versioned YAML next to the metrics, grouping, inhibition, silences, routing trees; Grafana alerting shown as the alternative (§7.2).
- *Why Docker Compose and not Kubernetes?* — single-host demo, one command, identical concepts; K8s path = kube-prometheus-stack (§9.2).
- *Why MLflow?* — OSS, self-hosted, tracking + registry + packaging; the other team's artifact is MLflow-packaged already (§8.2).
- *Why write a custom drift exporter instead of Evidently/whylogs?* — assignment asks for it; small, transparent, real-time, per-feature Prometheus gauges; Evidently could replace the math inside `compute()` without changing interfaces (§5.10).
- *Why a gateway/sidecar and not instrumenting the model?* — zero code changes, language-agnostic, measures what the client experiences; in-process shown as the alternative (§4.5).

Metrics & PromQL

- *Counter vs gauge vs histogram vs summary?* — §3.4; histogram vs summary: aggregatable, quantiles computed server-side (§3.5).
- *p95 vs average?* — tails hide in averages; percentiles = user experience (§3.5).
- *rate vs irate vs increase?* — averaged per-second / last two samples / total in window (§3.6).
- *avg vs avg_over_time?* — across series now vs one series across time.
- *What does [5m] mean vs the dashboard time range?* — smoothing window per point vs displayed period.
- *Pull vs push?* — §3.3; health detection & central config vs short-lived jobs.
- *What is a recording rule?* — pre-computed PromQL under a new name; one definition of p95, cheaper dashboards.
- *Cardinality?* — every label combination is a series; bounded labels only.
- *What is an exporter?* — anything serving `/metrics`; three kinds: library, sidecar, stand-alone.

Drift

- *Data vs concept vs prediction drift?* — $P(X) / P(Y|X) / P(\hat{y})$; labels needed only for concept (§5.1).
- *PSI formula & thresholds?* — $\sum(a-e) \cdot \ln(a/e)$; 0.10 / 0.25 (§5.3).
- *KS test?* — max CDF distance D + p-value; numeric only; we require $D > 0.2$ (§5.3).
- *Why both?* — PSI covers categoricals & is interpretable; KS is a proper test without bins.
- *Why a rolling window? how big?* — statistical power $\propto n$; 500 rows $\approx$ 2 min at demo traffic; bigger = fewer false alarms, slower (§5.4).
- *What is the reference?* — training-data profile: quantile bins, category shares, sample, score histogram; regenerate on retrain (§5.5).
- *What if labels arrive later?* — add an accuracy exporter the same way (§10.3).

Alerting

- *pending vs firing?* `for:` ? — state, not samples (§7.3).
- *Grouping / inhibition / silence?* — §7.5.
- *Operational vs ML-quality alerts?* — who gets woken up (§7.6).
- *How are Slack/e-mail configured?* — receivers + routes; env-substituted; demo sinks Mailhog & webhook catcher (§7.4).
- *Runbooks?* — annotation `runbook_url` → §12.

Model & story

- *What model is this?* — other team's Telco churn GradientBoosting (Optuna-tuned, MLflow, skops), FastAPI.
 - *What did you change in it?* — fixed transformer order (AUC 0.53 → 0.94 on train data), added ML-level metrics and a demo-only `/chaos` endpoint (§4.7).
 - *How would the stack have caught their bug in production?* — prediction distribution / positive rate far from the reference → PredictionDrift & PositiveRateAnomaly while ops signals stay green.
 - *How do I wire my model?* — env vars on a gateway + one scrape job (§10), reference JSON for drift.
 - *What is NOT covered?* — logs/traces (Loki/Tempo/OTel), long-term storage (Thanos/Mimir), authentication, HA, accuracy-with-labels, automated retraining trigger — all listed as next steps (§14).
-

15 · Hardware & saturation — the psutil system exporter

15.1 Concept

Saturation is the fourth golden signal: how "full" the machine is. Latency and error alerts tell you **that** something is wrong; CPU / memory / disk tell you **why** (and warn before it happens). Model containers are memory-heavy (the churn API alone holds ~370 MB), Prometheus and MLflow write to disk continuously — a full disk silently stops history and artefacts.

15.2 Why psutil (and not node_exporter / cAdvisor)

Option	Notes
psutil exporter (chosen) — <code>services/system_exporter/system_exporter.py</code>	120 lines of Python; host CPU (total + per core), load, memory, swap, disk, network counters, process count; plus per-container CPU % and memory via the Docker Engine API when <code>/var/run/docker.sock</code> is mounted (the numbers <code>docker stats</code> shows, as time series). Portable to any OS the stack runs on.
<code>node_exporter</code>	the standard on Linux hosts; inside Docker Desktop (macOS/Windows) it reports the Linux VM and needs host mounts that misbehave on Apple Silicon
<code>cAdvisor</code>	container metrics; same VM problem, heavy image, unstable on arm64 Docker Desktop
Datadog / cloud agents	SaaS
On a Linux server you would run <code>node_exporter</code> + <code>cAdvisor</code> and keep this exporter for the psutil-only extras; the alerts and	
panels are written so both sources can co-exist.	

15.3 Metrics

```
sys_cpu_percent{mode="total"} · sys_cpu_percent_per_core{core} · sys_load_average{period} ·  
sys_memory_bytes{kind} · sys_memory_percent · sys_swap_bytes{kind} ·  
sys_disk_bytes{mount,kind} · sys_disk_percent{mount} · sys_net_bytes_total{direction} ·  
sys_process_count · sys_cpu_cores · container_cpu_percent{name} ·  
container_memory_bytes{name} · container_memory_limit_bytes{name} ·  
sys_exporter_last_sample_timestamp.
```

15.4 PromQL

```
sys_cpu_percent{mode="total"}           # host CPU now  
avg_over_time(sys_cpu_percent{mode="total"}[5m])  # smoothed — what the alert  
uses  
sys_memory_percent                       # RAM %  
sys_disk_percent{mount="/" }  
rate(sys_net_bytes_total{direction="recv"}[1m])  # bytes/s in
```

```
topk(3, container_memory_bytes)          # heaviest containers
container_memory_bytes / container_memory_limit_bytes  # share of cgroup limit
```

15.5 Alerts (group `resource_alerts`)

`HostHighCPU` (avg > 85 % for 5 m, warning) · `HostHighMemory` (> 90 % for 5 m, warning) · `HostDiskAlmostFull` (> 90 % for 10 m, **critical**) · `ContainerHighMemory` (> 90 % of the container's limit, warning). Routed like every other operational alert (severity → receiver).

15.6 Dashboard

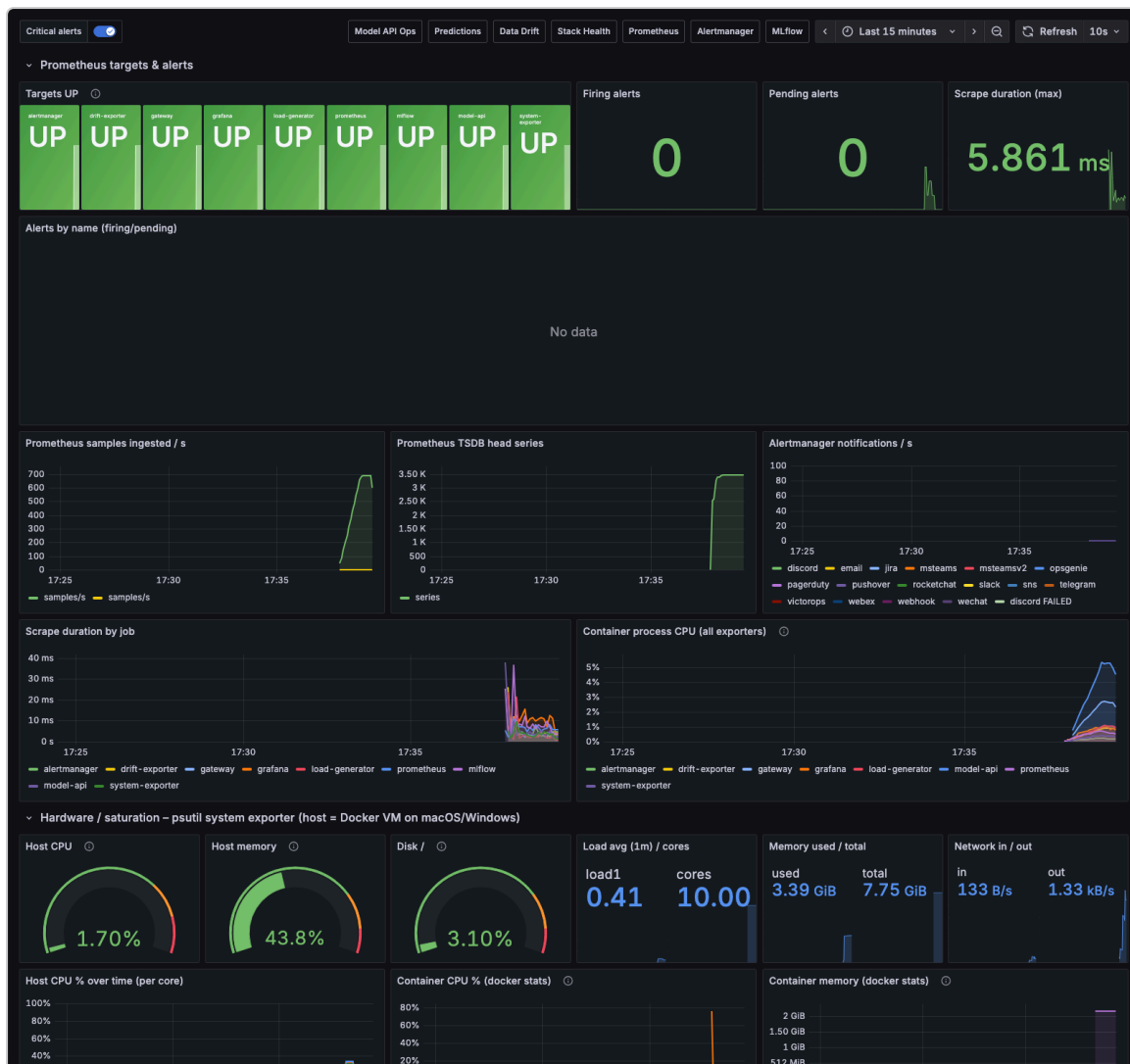
Stack-Health dashboard, row "*Hardware / saturation*": gauges for host CPU / memory / disk, load-average vs cores, memory used/total, network in/out, per-core CPU over time, per-container CPU % and memory.

15.7 Demo & troubleshooting

`make scenario-burst` (40 rps) → watch host CPU and the model-api container CPU rise; `docker compose logs system-exporter` prints "docker stats unavailable" if the socket is not mounted (host metrics still work). Inside Docker Desktop the "host" numbers describe the Linux VM (its cores/RAM as configured in Docker Desktop settings), not the Mac itself — say so in the demo.

15.8 Questions you may be asked

Why psutil? — portable, tiny, and gives per-container stats through the Docker socket · *Difference to node_exporter?* — same idea, `node_exporter` is the Linux standard with hundreds of collectors; ours is what fits a Docker-Desktop demo · *Why alert on avg_over_time?* — one busy scrape is not saturation · *What is saturation vs utilisation?* — utilisation is "how busy", saturation is "how much work is queued"; load average vs cores approximates the latter.



Stack-Health dashboard with the psutil hardware row (host CPU 1.7 %, RAM 43.8 %, disk 3.1 %, per-container CPU/memory).

14 · Glossary, next steps & references

Glossary

Term	Meaning
Alertmanager	Prometheus component that groups, routes, inhibits, silences alerts and sends notifications
Annotation (Grafana)	event marker overlaid on graphs (we overlay critical alerts)
Cardinality	number of distinct time series (label combinations)
Concept drift	change in $P(Y X)$; needs labels
Counter / Gauge / Histogram / Summary	Prometheus metric types (§3.4)
Data drift (covariate shift)	change in $P(X)$; label-free
Exporter	any process serving Prometheus text on <code>/metrics</code>
Golden signals	latency, traffic, errors, saturation (Google SRE)
Inhibition	rule that mutes alerts B while alert A fires
Instant vs range query	one value per series at a time (<i>table</i>) vs values over a period (<i>graph</i>)
KS test	Kolmogorov–Smirnov two-sample test; $D = \max \text{CDF gap}$
Model registry	versioned catalogue of models with aliases/stages (MLflow)
Observability	ability to answer unanticipated questions from telemetry (metrics/logs/traces)
p50/p95/p99	percentiles of latency; p95 = 95 % of requests are faster
Prediction drift	change in the model's output distribution
PromQL	Prometheus query language
PSI	Population Stability Index $\Sigma(a-e) \cdot \ln(a/e)$
Pull vs push	scraper fetches metrics vs apps send them
Recording rule	pre-computed PromQL saved as a new series
Reference profile	training-data distribution summary used by the drift exporter
Rolling window	last N observations kept in memory for drift statistics
Scrape	one HTTP fetch of <code>/metrics</code> by Prometheus
SLI / SLO / SLA	indicator / objective / agreement
Sidecar / gateway	proxy that observes traffic to an un-instrumented service
Silence	temporary mute of matching alerts (maintenance)
Target / job	a scrape endpoint / a group of them in <code>prometheus.yml</code>
TSDB	time-series database (Prometheus' storage)

Next steps (if the project continued)

1. **Logs & traces** — Loki (logs) + Tempo/Jaeger (traces) via OpenTelemetry; correlate a slow p99 with a trace.
2. **Long-term storage / HA** — remote-write to Thanos, Mimir or VictoriaMetrics; two Prometheus replicas; Alertmanager cluster.
3. **Accuracy exporter** — join delayed labels to stored predictions → precision/recall/AUC over time; concept-drift alerts.
4. **Segmented drift** — per contract type / region label on drift metrics.
5. **Automated retraining loop** — Alertmanager webhook → trainer → MLflow comparison → promotion gate → redeploy → new reference.
6. **Kubernetes** — kube-prometheus-stack, ServiceMonitors, dashboards as ConfigMaps, HPA on `ml:request_rate:1m`.
7. **Security** — Grafana SSO, Prometheus behind auth proxy, secrets for Slack/SMTP.
8. **SLO tooling** — Sloth / Pyrra for multi-window burn-rate alerts instead of static thresholds.

References

- Prometheus docs — <https://prometheus.io/docs/> (querying basics, histograms & summaries, alerting rules)
- Alertmanager docs — <https://prometheus.io/docs/alerting/latest/alertmanager/>
- Grafana docs — <https://grafana.com/docs/grafana/latest/> (provisioning, dashboards, alerting)
- MLflow docs — <https://mlflow.org/docs/latest/> (tracking, model registry, models)

- Google SRE Book, ch. 6 "Monitoring Distributed Systems" (golden signals) — <https://sre.google/sre-book/monitoring-distributed-systems/>
- Population Stability Index — Yurdakul, B. (2018) *Statistical properties of PSI*; credit-scoring practice thresholds 0.1/0.25
- Kolmogorov–Smirnov test — `scipy.stats.ks_2samp` documentation
- Evidently AI docs (drift methods overview) — <https://docs.evidentlyai.com/>
- Docker Compose docs — <https://docs.docker.com/compose/>
- Other team's model repository — <https://github.com/Nisarg7314/customer-churn-mlops>
- This repository — <https://github.com/AutomationArtist01/MLOPS>

Built by Team 4. Everything in this book is reproducible with `make up` & `make train`.

Part II — Pipeline, Tuning, CI/CD & Deployment

What was added after the grading rubric review — and why each tool was chosen over its alternatives

ZenML · Optuna · pytest + Ruff · GitHub Actions · Render · uv pip compile

Same recipe as Part I: Concept → Why it matters → Available tools → Alternatives & trade-offs → Why we chose ours → Exactly what we did (steps) → How to run / verify → Troubleshooting → Questions you may be asked

15 · What changed and why (the rubric pass)

15.1 Trigger

The course grading rubric (100 points, 8 categories, applies to all 15 projects) scores things the Project-4 brief did not mention: a **ZenML** pipeline (15), **MLflow + Optuna** tuning with registry promotion (15), **Docker + Render** deployment (15), **GitHub Actions** CI/CD (10), repository hygiene (10) and a report in a fixed structure (10). Part I of this book covers the monitoring stack (15) and problem framing (10). This part covers everything added to close the gap. Every addition stays on the **same Telco churn model** — no scope creep to other models.

15.2 Steps performed, in order

#	Step	Where	Result
1	Reorganised the repository into <code>services/</code> (things that run as containers), <code>monitoring/</code> (Prometheus/Grafana/Alertmanager config), <code>external/</code> (other team's repo snapshot + <code>PROVENANCE.md</code>), <code>docs/</code> , <code>tests/</code>	repo root	rubric §7 "clear structure"
2	Wrote a ZenML pipeline with 8 steps and a data-validation gate	<code>services/trainer/pipeline.py</code> , <code>run_pipeline.py</code>	rubric §2
3	Replaced the grid search with Optuna TPE + MedianPruner, 25 trials, nested MLflow runs, quality-gated Model Registry promotion (<code>@champion</code> , <code>stage=production</code>)	same file	rubric §3
4	Made the pipeline write the drift reference for the exporter, so monitoring is regenerated with every retrain	<code>build_drift_reference</code> step	closes the loop between training and monitoring
5	Pinned every dependency with <code>uv pip compile</code> (<code>requirements.in</code> → <code>requirements.txt</code> , 7 services)	<code>services/*/requirements*.txt</code>	rubric §7
6	Added unit tests (drift maths, gateway helper, reference builder, validation gate, config sanity) + kept the 23-assertion smoke test	<code>tests/test_units.py</code> , <code>tests/smoke_test.py</code>	rubric §5/§7
7	Added Ruff lint config + <code>pyproject.toml</code> , <code>requirements-dev.txt</code>	repo root	rubric §7
8	Added GitHub Actions workflow: lint → tests → promtool/amtool → compose validation → dashboards reproducible → build 6 images (push to GHCR on main) → compose smoke test	<code>.github/workflows/ci.yml</code>	rubric §5
9	Added a Render blueprint for the model API (Docker web service, <code>/health</code> check, <code>CHAOS_ENABLED=0</code>) and made the image <code>PORT</code> -aware	<code>render.yaml</code> , <code>services/model_api/Dockerfile</code>	rubric §4

#	Step	Where	Result
10	Rewrote the README with problem statement, dataset justification, success-metric table, hour budget, pipeline & deployment sections	README.md	rubric §1
11	Made 21 granular git commits (scaffolding, data, external, each service, monitoring, CI, docs) instead of one dump	git history	rubric §7
12	Wrote the project report in the rubric's structure with real screenshots (ZenML log, MLflow runs/chart/registry, Grafana x3, Alertmanager, notifications)	docs/Team4-Project-Report.pdf (source docs/report/)	rubric §8
13	Ran everything end-to-end: pipeline (54 s, 25 trials), make smoke 23/23, make lint clean, make test 11 passed	—	evidence, not claims

Commands that reproduce it: `make up` → `make train` → `make test` → `make lint` → `make smoke` → `make report`.

15.3 Rubric position after the pass (honest estimate)

Category	Now	Missing for full marks
1 Problem & planning	9/10	—
2 ZenML	13/15	a ZenML dashboard screenshot (<code>zenml login --local</code>) instead of the run log
3 MLflow + Optuna	14/15	—
4 Docker + Render	10/15 → 14	the live Render URL (needs the team's Render account: New → Blueprint → this repo → Apply)
5 GitHub Actions	4/10 → 9	push the repo so the workflow runs and shows green
6 Monitoring	15/15	—
7 Repo hygiene	8/10	commits spread over several days (keep committing small changes until submission)
8 Report	8/10 → 10	Actions + Render screenshots once the two actions above are done (<code>make report</code>)

16 · ZenML — pipeline orchestration

16.1 Concept

An **orchestrator** turns a training script into a graph of **steps** with explicit inputs/outputs. Each step's outputs become versioned **artifacts**; steps whose inputs did not change are **cached**; every execution is a **run** with lineage (which data → which model → which metrics). ZenML does this with two decorators — `@step` and `@pipeline` — and a pluggable **stack** (orchestrator, artifact store, experiment tracker, ...). Locally the stack is "run in-process + SQLite metadata + local artifact folder"; the *same code* can later run on Airflow, Kubeflow, Vertex or SageMaker by switching the stack, not the code.

16.2 Why it matters (why not a script?)

A script `train.py` works until: someone changes the CSV and nobody notices; you want to re-run evaluation without re-training; you need to prove which data produced the deployed model; you want the run to *refuse* bad data. Steps + validation gate + caching + lineage answer those. The rubric asks exactly for that: distinct reusable steps, validation that *halts* the pipeline, versioned/cached runs.

16.3 Alternatives & trade-offs

Tool	Type	Strengths	Why not here
ZenML (chosen)	ML-pipeline framework, stack abstraction	pure-Python decorators, caching, artifact versioning, model registry integration, local→cloud portability, tiny footprint for a demo	none blocking; heavier install (needs <code>zenml[local]</code> for the SQLite store)
Apache Airflow	general workflow scheduler (DAGs)	industry standard for batch scheduling, huge ecosystem	infra-heavy (scheduler, webserver, DB), DAG files not ML-aware (no artifact typing/caching), overkill for a laptop
Prefect / Dagster	modern Python orchestrators	great DX, Dagster has typed assets & lineage	not ML-specific (no experiment-tracker/model-registry stack), another server to run; rubric names ZenML
Kubeflow Pipelines	Kubernetes-native ML pipelines	scalable, componentised	needs a Kubernetes cluster; wrong scale for the demo
Metaflow (Netflix)	Python ML workflow	simple, versioned artifacts, cloud scaling	AWS-centric batch scaling; less integration with MLflow registry
MLflow Projects / Recipes	MLflow's own runners	already in the stack	Recipes are deprecated; no step caching/gating semantics
Makefile / plain script	—	zero deps	no lineage, caching, gating, or step reuse — the rubric's "Needs improvement" band

Why ZenML for us: the rubric requires it; it maps 1:1 to what we need (steps, cache, gate, MLflow alongside); it runs with no extra servers inside our Compose `trainer` container; and the pipeline code

stays portable if the team ever moves training to a cluster.

16.4 What we implemented (`services/trainer/pipeline.py`)

Step	Input → Output	Notes
<code>ingest_data(csv_path)</code>	path → <code>raw_data</code> DataFrame	7 043 × 21
<code>validate_data(df)</code>	→ <code>validated_data</code>	gate : 23 required columns present; <code>TotalCharges</code> coerced to numeric; null share ≤ 5 % per column; target ∈ {Yes, No}; minority class ≥ 10 %; tenure ≥ 0; ≥ 1 000 rows. Prints a JSON report; raises <code>ValueError</code> → the run halts
<code>split_data(df)</code>	→ <code>X_train</code> , <code>X_test</code> , <code>y_train</code> , <code>y_test</code>	stratified 80/20, seed 42, target → 0/1
<code>tune_and_train(X_train, y_train, n_trials)</code>	→ <code>model</code> (sklearn Pipeline), <code>best_params</code> , <code>mlflow_run_id</code>	Optuna study (Ch. 17); <code>enable_cache=False</code> because tuning must actually run
<code>evaluate_model(model, X_test, y_test, ...)</code>	→ <code>test_metrics</code>	AUC/acc/precision/recall/F1 logged to the parent MLflow run + model artifact with signature & input example
<code>register_model(run_id, metrics)</code>	→ <code>model_version</code>	quality gate AUC ≥ 0.80 → registry <code>telco-churn-classifier</code> , alias <code>champion</code> , tags <code>stage=production</code> , <code>test_roc_auc</code> , <code>validated_by</code>
<code>build_drift_reference(model, X_train, version)</code>	→ path	quantile bins/probs/sample per numeric feature, category shares, training-score histogram → <code>artifacts/telco_reference.json</code> (copy into <code>services/drift_exporter/reference/</code> to roll out)
<code>export_runs_csv(version)</code>	→ path	<code>mlflow.search_runs(...).to_csv</code>

`@pipeline(enable_cache=True) churn_training_pipeline(csv_path, n_trials)` wires them. Artifacts are typed with `Annotated[... , "name"]` so they show up by name in ZenML's store; pandas/sklearn objects are handled by ZenML's built-in materializers.

16.5 How to run / verify

```
make train                # docker compose run --rm trainer (25 trials, ~1 min on
a laptop)
make train-quick          # 8 trials
docker compose run --rm trainer zenml pipeline runs list          # run history (persisted in
the zenml_data volume)
docker compose run --rm trainer zenml login --local              # optional dashboard
pytest tests/test_units.py -k validation                         # the gate halts on a
missing column / imbalance
```


Observed run: validate report {"rows": 7043, "null_share": {"TotalCharges": 0.0016}, "churn_rate": 0.265, "problems": []} → 25 trials (8 pruned) → hold-out AUC 0.847 → registry v3 → reference written → 70 runs exported; 54 s total.

```
$ docker compose run --rm trainer -- python run_pipeline.py (ZenML · churn_training_pipeline · 25 Optuna trials)

Initiating a new run for the pipeline: churn_training_pipeline.
Registered new pipeline: churn_training_pipeline.
Step ingest_data has started.
[ingest] /data/telco_churn.csv: 7043 rows × 21 cols
Step ingest_data has finished in 0.317s.
Step validate_data has started.
[validate] ("rows": 7043, "null_share": {"TotalCharges": 0.0016}, "churn_rate": 0.2653698707936959, "problems": [])
Step validate_data has finished in 0.211s.
Step split_data has started.
[split] train 5634 / test 1409 · churn rate 0.265
Step split_data has finished in 0.814s.
Step tune_and_train has started.
[I 2026-08-18 06:36:51.491] Trial 0 finished with value: 0.8644262139885077 and parameters: {'n_estimators': 150, 'learning_rate': 0.2536999076681772, 'max_depth': 5, 'min_samples_split': 13, 'min
[I 2026-08-18 06:36:52.495] Trial 1 finished with value: 0.8391996163136096 and parameters: {'n_estimators': 50, 'learning_rate': 0.19030368301735815, 'max_depth': 5, 'min_samples_split': 15, 'min
[I 2026-08-18 06:36:54.316] Trial 2 finished with value: 0.8494733209940802 and parameters: {'n_estimators': 275, 'learning_rate': 0.020589720197687916, 'max_depth': 2, 'min_samples_split': 5, 'min
[I 2026-08-18 06:36:56.335] Trial 3 finished with value: 0.8477233826222745 and parameters: {'n_estimators': 150, 'learning_rate': 0.02692655251486473, 'max_depth': 5, 'min_samples_split': 4, 'min
[I 2026-08-18 06:36:57.399] Trial 4 finished with value: 0.8450321173641977 and parameters: {'n_estimators': 175, 'learning_rate': 0.14447746112718687, 'max_depth': 2, 'min_samples_split': 11, 'min
[I 2026-08-18 06:36:58.910] Trial 5 finished with value: 0.8464535649052619 and parameters: {'n_estimators': 200, 'learning_rate': 0.0178601378893971, 'max_depth': 2, 'min_samples_split': 20, 'min
[I 2026-08-18 06:37:00.732] Trial 6 finished with value: 0.848042823043914 and parameters: {'n_estimators': 125, 'learning_rate': 0.013940346079873234, 'max_depth': 5, 'min_samples_split': 10, 'min
[I 2026-08-18 06:37:01.240] Trial 7 pruned.
[I 2026-08-18 06:37:03.779] Trial 8 pruned.
[I 2026-08-18 06:37:04.721] Trial 9 pruned.
[I 2026-08-18 06:37:06.678] Trial 10 pruned.
[I 2026-08-18 06:37:10.081] Trial 11 finished with value: 0.8491799620920876 and parameters: {'n_estimators': 300, 'learning_rate': 0.010446609340209814, 'max_depth': 4, 'min_samples_split': 8, 'min
[I 2026-08-18 06:37:12.901] Trial 12 finished with value: 0.8492350343068101 and parameters: {'n_estimators': 300, 'learning_rate': 0.01101278928574014, 'max_depth': 3, 'min_samples_split': 7, 'min
[I 2026-08-18 06:37:15.307] Trial 13 finished with value: 0.8484675964297291 and parameters: {'n_estimators': 250, 'learning_rate': 0.037370046395123875, 'max_depth': 3, 'min_samples_split': 7, 'min
[I 2026-08-18 06:37:16.997] Trial 14 pruned.
[I 2026-08-18 06:37:19.664] Trial 15 finished with value: 0.8494848009036694 and parameters: {'n_estimators': 250, 'learning_rate': 0.010088777799749494, 'max_depth': 4, 'min_samples_split': 2, 'min
[I 2026-08-18 06:37:22.344] Trial 16 finished with value: 0.8483195808017291 and parameters: {'n_estimators': 250, 'learning_rate': 0.02604358152505174, 'max_depth': 4, 'min_samples_split': 2, 'min
[I 2026-08-18 06:37:24.117] Trial 17 pruned.
[I 2026-08-18 06:37:27.045] Trial 18 pruned.
[I 2026-08-18 06:37:29.366] Trial 19 finished with value: 0.8492084233970817 and parameters: {'n_estimators': 225, 'learning_rate': 0.014525573478714747, 'max_depth': 4, 'min_samples_split': 10, 'min
[I 2026-08-18 06:37:30.905] Trial 20 finished with value: 0.8497844935388331 and parameters: {'n_estimators': 275, 'learning_rate': 0.03543965171315069, 'max_depth': 2, 'min_samples_split': 5, 'min
[I 2026-08-18 06:37:32.435] Trial 21 finished with value: 0.8495458022372618 and parameters: {'n_estimators': 275, 'learning_rate': 0.03892786471849319, 'max_depth': 2, 'min_samples_split': 5, 'min
[I 2026-08-18 06:37:33.533] Trial 22 pruned.
[I 2026-08-18 06:37:34.884] Trial 23 finished with value: 0.8497535440230962 and parameters: {'n_estimators': 225, 'learning_rate': 0.03704001852011789, 'max_depth': 2, 'min_samples_split': 8, 'min
[I 2026-08-18 06:37:36.237] Trial 24 finished with value: 0.8499492578162591 and parameters: {'n_estimators': 225, 'learning_rate': 0.03789002839130302, 'max_depth': 2, 'min_samples_split': 9, 'min
[Optuna] 25 trials (8 pruned) · best cv AUC 0.8499 · {'n_estimators': 225, 'learning_rate': 0.03789002839130302, 'max_depth': 2, 'min_samples_split': 9, 'min_samples_leaf': 9, 'subsample': 0.645642
Step tune_and_train has finished in 47.723s.
Step evaluate_model has started.
[evaluate] {"test_roc_auc": 0.847, "test_accuracy": 0.8048, "test_precision": 0.6701, "test_recall": 0.5214, "test_f1": 0.5865}
Step evaluate_model has finished in 3.898s.
Step register_model has started.
Created version '3' of model 'telco-churn-classifier'.
[registry] telco-churn-classifier v3 → @champion (stage=production)
Step register_model has finished in 0.714s.
Step build_drift_reference has started.
[reference] wrote /artifacts/telco_reference.json
Step build_drift_reference has finished in 0.054s.
Step export_runs_csv has started.
[csv] 70 runs → /artifacts/mlflow_runs.csv
Step export_runs_csv has finished in 0.109s.
Pipeline run has finished in 54.078s.
[zenml] pipeline run: churn_training_pipeline-2026_08_18-06_36_47_919531 · status=completed
```

ZenML pipeline run log (steps, validation report, Optuna trials, registry promotion)

16.6 Troubleshooting

- `ModuleNotFoundError: sqlalchemy_utils` → the local SQLite store needs the extra: `zenml[local]`.
- `zenml init` inside the image so `python run_pipeline.py` needs no interactive setup; `ZENML_ANALYTICS_OPT_IN=false` silences telemetry prompts.
- First `pip install zenml` is large; behind a slow network build once and reuse the image.
- Steps returning several values must declare a `tuple[Annotated[...], ...]` return type.

16.7 Questions you may be asked

- *ZenML vs Airflow?* — Airflow schedules generic DAGs; ZenML models ML steps with typed, versioned artifacts and caching and can *run on* Airflow as an orchestrator.
- *What does the validation step check and what happens on failure?* — schema, nulls, target values, class balance, ranges; it raises, so nothing downstream runs (no model gets registered).
- *What is cached?* — every step except tuning; change the CSV → ingest/validate re-run; change only `n_trials` → earlier steps are served from cache.
- *Where is lineage stored?* — ZenML SQLite metadata (volume `zenml_data`) + MLflow (runs, model).

17 · Optuna — hyper-parameter optimisation

17.1 Concept

Optuna is a **define-by-run** hyper-parameter optimisation framework: the objective function *asks* a `trial` for values (`trial.suggest_int/float/categorical`), a **sampler** proposes the next values (default **TPE**, Tree-structured Parzen Estimator — a Bayesian approach that models "good" vs "bad" regions), and a **pruner** stops unpromising trials early using intermediate results (`trial.report(value, step)` / `trial.should_prune()`).

17.2 Why it matters

Grid search wastes trials on obviously bad corners and scales exponentially with parameters; random search is better but blind. TPE spends later trials near promising regions, and pruning cuts the cost of bad ones — in our run 8 of 25 trials were pruned after the first CV fold. It also gives *insight* (parameter importances, history plots) rather than just a winner.

17.3 Alternatives & trade-offs

Tool	Notes	Why not here
Optuna (chosen)	TPE/CMA-ES/random samplers, pruners, storage, dashboards, framework-agnostic, ~1 dependency	—
<code>GridSearchCV</code> (sklearn)	exhaustive, simple	exponential cost, no early stopping, no learning from history — what v1 of our trainer did
<code>RandomizedSearchCV</code>	cheap, surprisingly strong	no pruning, no adaptivity
Hyperopt	original TPE library	less maintained, clunkier API, weaker pruning
Ray Tune	distributed tuning, many schedulers (ASHA, PBT)	needs Ray cluster; overkill for one laptop
scikit-optimize / <code>BayesSearchCV</code>	Gaussian-process BO	slower per trial, less active project
Keras Tuner / Ax-BoTorch	framework-specific / research-grade	wrong fit for sklearn GBM

Why Optuna: the rubric names it; the other team used it (comparability); best DX for sklearn; TPE + MedianPruner give real tuning insight at 25 trials; integrates trivially with MLflow (nested runs).

17.4 What we implemented

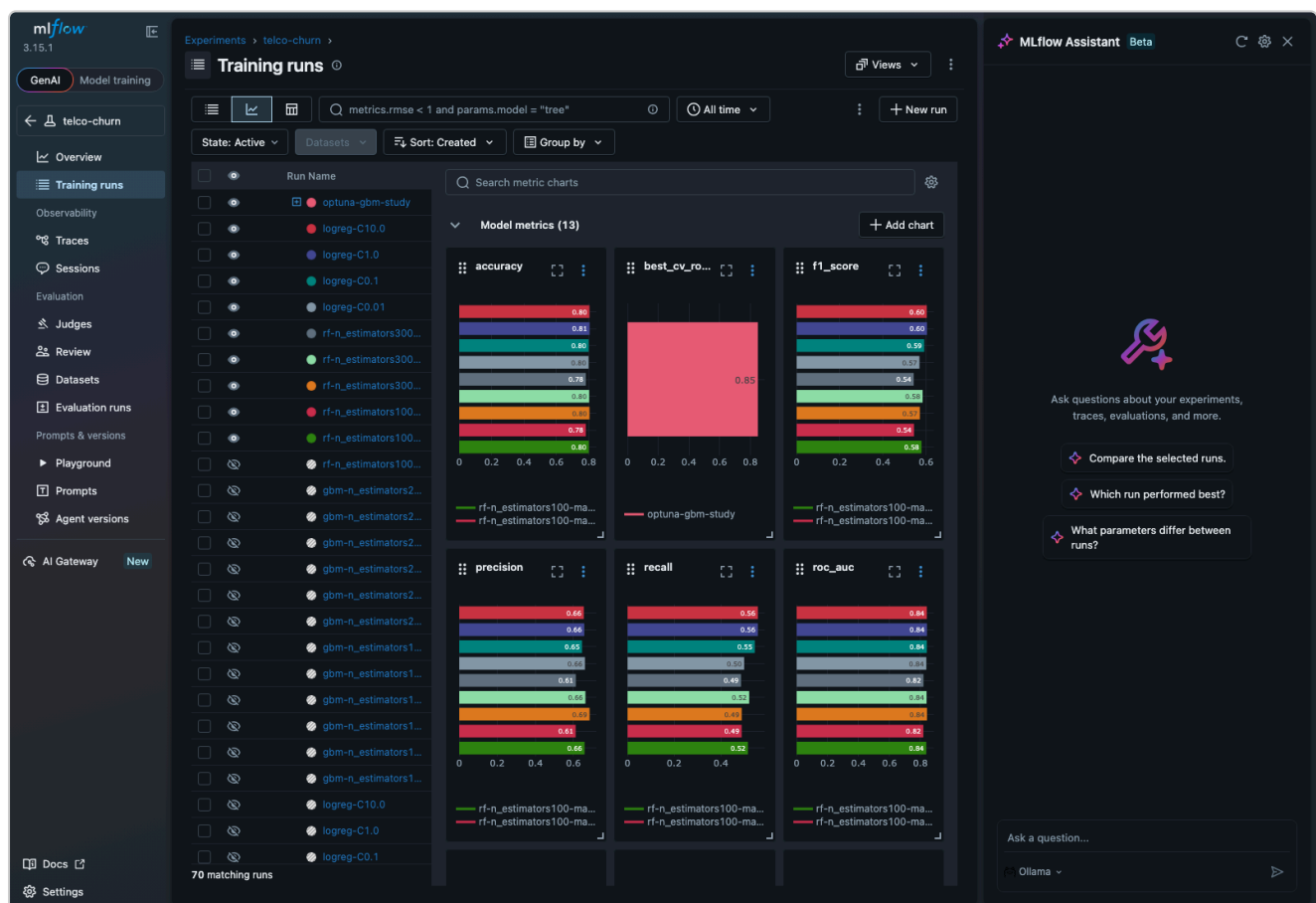
- Search space (GradientBoosting): `n_estimators` 50–300 step 25 · `learning_rate` 0.01–0.3 (log) · `max_depth` 2–6 · `min_samples_split` 2–20 · `min_samples_leaf` 1–10 · `subsample` 0.6–1.0.
- Objective: mean **3-fold stratified CV ROC-AUC**; after each fold `trial.report()` → MedianPruner (`n_startup_trials=5`, `n_warmup_steps=1`).

- Sampler `TPESampler(seed=42)` for reproducibility; `study.optimize(n_trials=25)`.
- **MLflow**: parent run `optuna-gbm-study` (tags `sampler/pruner`, params `n_trials/cv_folds/n_train`, metrics `best_cv_roc_auc`, `trials_pruned`, artifacts `optuna_trials.json` and `optuna_param_importance.json`); each trial a **nested run** with its params, `cv_roc_auc`, `cv_roc_auc_std`, `fit_seconds`, `pruned` flag and state tag.
- Best config retrained on the full training split → evaluated → registered (Ch. 16).

Result: best CV AUC **0.8499** (`n_estimators=225`, `lr=0.038`, `depth=2`, `min_split=9`, `min_leaf=9`, `subsample=0.65`), hold-out **AUC 0.847 / F1 0.587 / precision 0.670 / recall 0.521 / accuracy 0.805**.
Insight from the trials: shallow trees (depth 2) with a small learning rate and ~200+ estimators win; depth ≥ 5 over-fits (trial 0: 0.804).

17.5 How to demo in MLflow (whiteboard items)

Runs table → expand `optuna-gbm-study` (nested trials) → filter `metrics.cv_roc_auc > 0.849` or `params.max_depth = '2'` → **Chart view** → parallel coordinates (`learning_rate`, `max_depth`, `n_estimators` → `cv_roc_auc`) → open the parent run → **Artifacts** → `optuna_param_importance.json`, `model/MLmodel` → **Models** → `telco-churn-classifier v3 @champion` → *Download CSV*.



MLflow chart view of the experiment (parent Optuna study + earlier grid runs)

17.6 Questions you may be asked

- *Why TPE?* — models the distribution of good vs bad params and samples where the ratio is highest; needs few trials to beat random search.

- *Why a pruner? which one?* — MedianPruner stops a trial when its running score is below the median of previous trials at the same step; cheap and robust.
- *Why 25 trials?* — rubric ≥ 20 ; each trial ≈ 2 s here, so more would be free — increase `OPTUNA_TRIALS` in `docker-compose.yml`.
- *Why CV inside the objective and a separate hold-out?* — tuning must never see the test set.
- *How is the best trial tied to the registered model?* — the parent run holds `best_*` params and the logged model; `register_model` registers `runs:<parent>/model`.

18 · Tests, lint and CI/CD — pytest · Ruff · GitHub Actions

18.1 Concept

Continuous integration = every push/PR runs the same automated checks; failures block merges. Layers here: **lint** (style/bugs without running code) → **unit tests** (fast, no Docker) → **config validation** (Prometheus/Alertmanager/Compose files are code too) → **image builds** → **integration smoke test** (boot the whole stack, assert behaviour).

18.2 Alternatives & trade-offs

Need	Chosen	Alternatives	Why
CI runner	GitHub Actions	GitLab CI, Jenkins, CircleCI, Travis, Azure Pipelines	repo is on GitHub → zero setup, free minutes, native GHCR push, rubric names it; Jenkins needs a server, GitLab CI needs GitLab
Test framework	pytest	unittest, nose2	fixtures, parametrisation, <code>importorskip</code> , plain asserts; de-facto standard
Lint	Ruff	flake8 + isort + pyupgrade, pylint, black	one Rust binary replacing all of them, 10–100× faster, same rule codes (E/F/W/I/B)
Config checks	promtool / amtool (official)	none	catch YAML/PromQL errors before deploy
Container registry	GHCR	Docker Hub, ECR	integrated with Actions token; no extra secrets
Integration test	compose smoke test in the runner	mocking everything	proves the stack actually boots and wires up

18.3 What we implemented

- `tests/test_units.py` (11 tests + 3 pipeline-gate tests that need ZenML): PSI zero/shift/edge cases, histogram normalisation, reference profile shape, **end-to-end drift detection on the exporter object** (baseline PSI < 0.25 → shifted tenure PSI > 0.25, KS p < 0.01), gateway dotted-path helper, `build_reference.py` CLI, validation gate pass/halt, Prometheus jobs present, every alert has severity+runbook+ for, dashboards valid JSON with panels.
- `pyproject.toml`: Ruff rules E/F/W/I/B (line length 120), pytest paths.
- `.github/workflows/ci.yml` — three jobs: **lint-and-test** (Ruff, pytest, promtool check config/rules, amtool check-config with env substituted, `docker compose config`, dashboards regenerated and `git diff --exit-code`), **docker-build** (matrix over 6 services, buildx cache, push to `ghcr.io/<owner>/team4-<service>` on main), **smoke-test** (`docker compose up -d --build`, wait for health, `python tests/smoke_test.py`, logs on failure, `down -v`).
- Locally: `make lint`, `make test`, `make smoke`.

18.4 Steps you still do

`git push origin main` → Actions tab shows the three jobs → screenshot into the report (docs/report/screenshots/github-actions.png , then `make report`).

18.5 Questions you may be asked

- *What blocks a merge?* — any failing job (branch protection rule on `main` recommended).
- *Why validate Prometheus rules in CI?* — a typo in an alert expression fails silently at runtime; `promtool` fails loudly at PR time.
- *Why build images in CI if you don't deploy from CI?* — proves Dockerfiles build from a clean machine; images in GHCR are what Render/Compose could pull.

19 · Deployment — Docker & Render

19.1 Concept

Container image = reproducible runtime; **Render** = PaaS that builds a Dockerfile from a Git repo and runs it as a web service with TLS, health checks and auto-restarts. A `render.yaml` **blueprint** declares the service so deployment is one click and reviewable in Git.

19.2 Alternatives & trade-offs

Platform	Notes	Why not here
Render (chosen)	free tier, Dockerfile-native, blueprint-as-code, health checks; the other team already deploys there; rubric names it	cold starts on free plan (~30 s)
Fly.io / Railway	similar PaaS, good DX	not in rubric; Fly needs CLI/credit card
Heroku	classic PaaS	no free tier any more
AWS App Runner / Google Cloud Run / Azure Container Apps	serverless containers, scale-to-zero	cloud accounts + billing; more setup than a demo needs
Kubernetes (EKS/GKE)	full control	far too heavy for one API
VM + docker compose	simplest "prod"	manual TLS/updates; no free tier

19.3 What we implemented

- `services/model_api/Dockerfile`: `python:3.12-slim`, requirements layer before code (cache), HEALTHCHECK on `/health`, CMD `uvicorn ... --port ${PORT:-8000}` (Render injects `PORT`).
- `render.yaml`: `type: web`, `runtime: docker`, `rootDir: services/model_api`, `healthCheckPath: /health`, `env` `MODEL_NAME`, `MODEL_VERSION`, `CHAOS_ENABLED=0` (no fault injection in public), `plan: free`.
- All 7 service images build in CI; Compose runs the same images locally.

19.4 Steps you still do (≈ 5 minutes)

1. Push the repo. 2. Render dashboard → **New** → **Blueprint** → pick the repo → **Apply**.
2. Wait for the build (first one ~5 min: sklearn wheels). 4. Open `https://<name>.onrender.com/health` → `{"status":"healthy",...}`; `/docs`; `/metrics`. 5. Put the URL in README §8 and take the screenshot for the report. 6. Optional: monitor the cloud instance from your laptop — `UPSTREAM_URL=https://<name>.onrender.com` on a second gateway (Ch. 10 wiring guide).

19.5 Questions you may be asked

- *Why not deploy Prometheus/Grafana to Render too?* — they need persistent disks and private networking; on Render's free tier that is impractical. The model API is the deployable unit; the monitoring stack runs where the team runs it (Compose now, Kubernetes later).
- *Health check?* — `/health` returns model name, version, feature count; Render restarts on failure; Prometheus/gateway use the same endpoint.

20 · Dependency pinning — `uv pip compile`

Every service has `requirements.in` (what we *want*: `fastapi==0.141.1`, `scikit-learn==1.9.0`, ...) and a generated `requirements.txt` (what we *install*: every transitive package with an exact version, annotated with who requires it). Regenerate: `uv pip compile requirements.in -o requirements.txt -p 3.12`.

Tool	Notes	Why not
<code>uv pip compile</code> (chosen)	pip-tools-compatible lock, extremely fast resolver, one binary	—
<code>pip-tools</code> (<code>pip-compile</code>)	same idea, slower	<code>uv</code> is a drop-in
<code>pip freeze</code>	snapshot of an env	includes junk, no "why", not reproducible from intent
Poetry / PDM / Hatch	full project managers with lock files	heavier; our services are tiny single-purpose containers
<code>conda-lock</code>	for conda envs	not needed (pure wheels)

Why it matters: the other team's artifact only loads with `scikit-learn==1.9.0` + `skops`; MLflow 3 changed host-header behaviour between minor versions; ZenML needs `zenml[local]`. Pins make the demo identical on every laptop and in CI (rubric §7 "dependencies pinned").

21 · The project report vs the rubric

`docs/Team4-Project-Report.pdf` follows the required order — objective → architecture → implementation → deployment → results → challenges → conclusion — with real screenshots (ZenML run log, MLflow runs table, chart view, registry, Grafana ops/drift/predictions, Alertmanager, notifications) and the repo link. Two evidence items are explicitly marked *pending* (GitHub Actions run, Render URL) rather than faked; add them and run `make report`.

Final checklist before submission

- ☐ `git push` → CI green → screenshot
- ☐ Render blueprint applied → URL in README + screenshot
- ☐ `make report` (recaptures screenshots, rebuilds the PDF)
- ☐ a few more small commits over the coming days (rubric §7 looks at commit spread)
- ☐ rehearse Ch. 11 demo + Ch. 13/16.7/17.6 questions